



Терминология

Пёс — игровой объект, способный перемещаться по игровой карте и взаимодействовать с другими объектами в соответствии с правилами игры.

Игровой сервер или просто сервер — программа, которая выполняется на серверном компьютере. Реализует игровую логику и предоставляет REST API.

Клиент — программное обеспечение, которое выполняется в браузере. Оно может взаимодействовать с сервером и визуализировать игровое состояние.

Пользователь — человек, участвующий в игре при помощи клиентского ПО.

Игрок — агент внутри игрового сервера, при помощи которого пользователь может управлять своим псом.

Токен — псевдослучайная последовательность символов, известная только пользователю и серверу. Поэтому пользователь может управлять лишь своим игроком, ведь токенов других игроков он не знает.

Реализуйте в игровом сервере операции входа в игру и получения информации об игроках. Пользователь может присоединиться к игре на любой из доступных карт. Для входа в игру он должен сообщить серверу кличку своего пса и id карты.

Сервер должен создать пса на указанной карте и игрока, управляющего псом, а затем вернуть клиенту id игрока и токен для аутентификации. Id нужен клиенту, чтобы отличать своего игрока от других. Клиент должен предъявить токен серверу, чтобы получать информацию о состоянии игрового сеанса и управлять своим игроком.

Вход в игру

Для входа в игру реализуйте обработку POST-запроса к точке входа `/api/v1/game/join`.

Параметры запроса:

Обязательный заголовок Content-Type должен иметь тип `application/json`.

Тело запроса — JSON-объект с обязательными полями `userName` и `mapId`: имя игрока и id карты. Имя игрока совпадает с именем пса.

Пример запроса:

`POST /api/v1/game/join HTTP/1.1`

`Content-Type: application/json`

```
{"userName": "Scooby Doo", "mapId": "map1"}
```

В случае успеха должен возвращаться ответ, обладающий следующими свойствами:

Статус-код 200 OK.

Заголовок Content-Type должен иметь тип `application/json`.

Заголовок Content-Length должен хранить размер тела ответа.

Обязательный заголовок `Cache-Control` должен иметь значение `no-cache`.

Тело ответа — JSON-объект с полями `authToken` и `playerId`. Поле `playerId` — целое число, задающее id игрока. Поле `authToken` — токен для авторизации в игре — строка, состоящая из 32 случайных шестнадцатеричных цифр.

Пример ответа:

HTTP/1.1 200 OK

Content-Type: application/json

Content-Length: 61

Cache-Control: no-cache

```
{"authToken":"6516861d89ebfff147bf2eb2b5153ae1","playerId":0}
```

Если в качестве mapId указан несуществующий id карты, должен вернуться ответ со статус-кодом 404 Not found и заголовками Content-Length: <размер тела ответа>, Content-Type: application/json и Cache-Control: no-cache. Тело ответа — JSON-объект с полями code и message. Поле code — строка "mapNotFound". Поле message — строка с понятным человеку текстом ошибки. Пример:

HTTP/1.1 404 Not found

Content-Type: application/json

Content-Length: 51

Cache-Control: no-cache

```
{"code": "mapNotFound", "message": "Map not found"}
```

Если было передано пустое имя игрока, должен вернуться ответ со статус-кодом 400 Bad request и заголовками Content-Length: <размер тела ответа>, Content-Type: application/json и Cache-Control: no-cache. Тело ответа — JSON-объект с полями code и message. Поле code — строка "invalidArgument". Поле message — строка с понятным человеку текстом ошибки. Пример:

HTTP/1.1 400 Bad request

Content-Type: application/json

Content-Length: 54

Cache-Control: no-cache

```
{"code": "invalidArgument", "message": "Invalid name"}
```

Если при парсинге JSON или получении его свойств произошла ошибка, должен вернуться ответ со статус-кодом 400 Bad request и заголовками Content-Length: <размер тела ответа>, Content-Type: application/json и Cache-Control: no-cache. Тело ответа — JSON-объект с полями code и message. Поле code — строка "invalidArgument". Поле message — строка с понятным человеку текстом ошибки. Пример:

HTTP/1.1 400 Bad request

Content-Type: application/json

Content-Length: 71

Cache-Control: no-cache

```
{"code": "invalidArgument", "message": "Join game request parse error"}
```

Если метод запроса отличается от POST, должен вернуться ответ со статус-кодом 405 Method Not Allowed и заголовками Content-Length: <размер тела ответа>, Content-Type: application/json, Allow: POST и Cache-Control: no-cache. Тело ответа — JSON-объект с полями code и message. Поле code — строка "invalidMethod", а message — строка с понятным человеку текстом ошибки. Пример:

HTTP/1.1 405 Method Not Allowed

Content-Type: application/json

Allow: POST

Content-Length: 68

Cache-Control: no-cache

```
{"code": "invalidMethod", "message": "Only POST method is expected"}
```

Получение списка игроков

Чтобы получить список игроков, находящихся в одной игровой сессии с игроком, используется GET-запрос к точке входа `/api/v1/game/players`. Параметры запроса: Обязательный заголовок `Authorization: Bearer <токен пользователя>`. В качестве токена пользователя следует передать токен, полученный при входе в игру. Этот токен сервер использует, чтобы аутентифицировать игрока и определить, на какой карте он находится.

Пример запроса:

GET `/api/v1/game/players` HTTP/1.1

Authorization: Bearer 6516861d89ebfff147bf2eb2b5153ae1

В случае успеха должен вернуться ответ со следующими свойствами:

Статус-код 200 OK.

Заголовок Content-Type: application/json.

Заголовок Content-Length: <размер тела ответа>.

Заголовок Cache-Control: no-cache.

Тело ответа — JSON-объект. Его ключи — идентификаторы пользователей на карте.

Значение каждого из этих ключей — JSON-объект с единственным полем `name`, строкой, задающей имя пользователя, под которым он вошёл в игру.

Пример ответа:

HTTP/1.1 200 OK

Content-Type: application/json

Content-Length: 75

Cache-Control: no-cache

```
{
  "0": {"name": "Harry Potter"},
  "1": {"name": "Hermione Granger"}
}
```

Если заголовок `Authorization` в запросе отсутствует либо его значение не удовлетворяет требованиям задачи, должен вернуться ответ со следующими свойствами:

Статус-код 401 Unauthorized.

Заголовок Content-Type: application/json.

Заголовок Content-Length: <размер тела ответа>.

Заголовок Cache-Control: no-cache.

Тело ответа — JSON-объект с полями `code` и `message`. Поле `code` должно иметь значение `"invalidToken"`, а `message` — содержать понятное человеку описание ошибки.

Пример:

HTTP/1.1 401 Unauthorized

Content-Type: application/json

Content-Length: 70

Cache-Control: no-cache

```
{"code": "invalidToken", "message": "Authorization header is missing"}
```

Если заголовок Authorization содержит валидное значение токена, но в игре нет пользователя с таким токеном, должен вернуться ответ со следующими свойствами: Статус-код 401 Unauthorized.

Заголовок Content-Type: application/json.

Заголовок Content-Length: <размер тела ответа>.

Заголовок Cache-Control: no-cache.

Тело ответа — JSON-объект с полями code и message.

Пример:

HTTP/1.1 401 Unauthorized

Content-Type: application/json

Content-Length: 70

Cache-Control: no-cache

```
{"code": "unknownToken", "message": "Player token has not been found"}
```

Если метод запроса отличается от GET или HEAD, должен вернуться ответ со следующими свойствами:

Статус-код 405 Method Not Allowed.

Заголовок Content-Type: application/json.

Заголовок Allow: GET, HEAD.

Заголовок Content-Length: <размер тела ответа>.

Заголовок Cache-Control: no-cache.

Тело ответа — JSON-объект с полями code и message.

Пример:

HTTP/1.1 405 Method Not Allowed

Content-Type: application/json

Allow: GET, HEAD

Content-Length: 54

Cache-Control: no-cache

```
{"code": "invalidMethod", "message": "Invalid method"}
```

Как будет тестироваться ваш код

В репозитории будут тесты, которые запустят ваш сервер, передав ему путь к конфигурационному файлу и путь к каталогу со статическими файлами. Тесты проверят, правильно ли сервер обрабатывает запросы к REST API и отдаёт статические файлы.

Рекомендации

Если класс RequestHandler (или его аналог в вашей программе) обрабатывает как запросы статических файлов, так и запросы к API, то сейчас самое время вынести обработку API запросов в отдельный класс. Ваш сервер будет обрабатывать всё больше и больше запросов, и RequestHandler станет сложнее поддерживать

Начиная с этого задания, ваш сервер будет не только возвращать состояние игры, но и изменять его. Бизнес-правила игры не должны зависеть от того, по какому протоколу происходит общение клиента с сервером. Это значит, что код, отвечающий за игровую логику, стоит отделить от кода, занимающегося обработкой HTTP-запросов. Так вы сделаете свой код проще.

Прежде чем писать код, рекомендуем спроектировать архитектуру. Правильно схему легче,

чем рефакторить уже написанный код. Обязательно сохраните нарисованные схемы — они вам пригодятся в будущих заданиях.

Нужно добавить два слоя: **игровую сессию/игроков** в модели и **API-обработчик**, который только валидирует HTTP/JSON и вызывает модель. В ваших файлах уже есть `Game` и заготовка `player.h`, а JSON-ответы централизуются через `MakeJsonResponse` / `MakeErrorResponse`, поэтому решение хорошо ложится в существующую архитектуру.^{[1] [2] [3]}

Архитектура

Предлагаю такую зависимость:



Ключевой момент: пользователь, вошедший на одну карту, должен видеть только игроков **той же карты**. Удобнее всего создать один `GameSession` на каждую `Map`.

Ваш `api_strand_` особенно важен: создание игроков, присвоение `id` и добавление токенов должны происходить последовательно, иначе при нескольких HTTP worker-потоках могут возникнуть гонки. В заготовке `RequestHandler` это уже предусмотрено: API-запросы можно выполнять через `net::dispatch(api_strand_, ...)`.^[3]

Модель игры

`player.h`

Ваш текущий `player.h` содержит две одноимённые декларации `PlayerTokens`, незавершённые типы `Dog`, `GameSession` и некорректное наследование `class Players : Player`. Его лучше заменить полноценным заголовком.

```
#pragma once

#include <array>
#include <stdint>
#include <iomanip>
#include <random>
#include <sstream>
```

```

#include <string>
#include <string_view>
#include <unordered_map>
#include <utility>
#include <vector>

#include "model.h"
#include "tagged.h"

namespace model {

namespace detail {
struct DogTag {};
struct PlayerTag {};
struct TokenTag {};
} // namespace detail

using DogId = util::Tagged<std::uint64_t, detail::DogTag>;
using PlayerId = util::Tagged<std::uint64_t, detail::PlayerTag>;
using Token = util::Tagged<std::string, detail::TokenTag>;

class Dog {
public:
    Dog(DogId id, std::string name)
        : id_(id)
        , name_(std::move(name)) {
    }

    DogId GetId() const noexcept {
        return id_;
    }

    const std::string& GetName() const noexcept {
        return name_;
    }

private:
    DogId id_;
    std::string name_;
};

class GameSession {
public:
    explicit GameSession(const Map* map)
        : map_(map) {
    }

    const Map& GetMap() const noexcept {
        return *map_;
    }

    Dog& AddDog(std::string name) {
        const auto id = DogId{next_dog_id++};
        return dogs_.emplace_back(id, std::move(name));
    }
}

```

```

        const std::vector<Dog>& GetDogs() const noexcept {
            return dogs_;
        }

private:
    const Map* map_;
    std::uint64_t next_dog_id_ = 0;
    std::vector<Dog> dogs_;
};

class Player {
public:
    Player(PlayerId id, GameSession& session, Dog& dog) noexcept
        : id_(id)
        , session_(&session)
        , dog_(&dog) {
    }

    PlayerId GetId() const noexcept {
        return id_;
    }

    const GameSession& GetSession() const noexcept {
        return *session_;
    }

    const Dog& GetDog() const noexcept {
        return *dog_;
    }

private:
    PlayerId id_;
    GameSession* session_;
    Dog* dog_;
};

class Players {
public:
    Player& Add(Dog& dog, GameSession& session) {
        const auto id = PlayerId{next_player_id_++};
        auto [it, inserted] = players_.emplace(
            id,
            Player{id, session, dog});
        return it->second;
    }

    Player* FindById(PlayerId id) noexcept {
        if (auto it = players_.find(id); it != players_.end()) {
            return &it->second;
        }
        return nullptr;
    }

    const std::unordered_map<
        PlayerId,
        Player,

```

```

        util::TaggedHasher<PlayerId>>& GetAll() const noexcept {
            return players_;
        }

private:
    std::uint64_t next_player_id_ = 0;

    std::unordered_map<
        PlayerId,
        Player,
        util::TaggedHasher<PlayerId>> players_;
};

class PlayerTokens {
public:
    PlayerTokens()
        : generator_(std::random_device{}()) {
    }

    Token AddPlayer(Player& player) {
        Token token;
        do {
            token = Token{GenerateToken()};
        } while (token_to_player_.contains(token));

        token_to_player_.emplace(token, &player);
        return token;
    }

    Player* FindPlayerByToken(const Token& token) const noexcept {
        if (auto it = token_to_player_.find(token);
            it != token_to_player_.end()) {
            return it->second;
        }
        return nullptr;
    }

private:
    std::string GenerateToken() {
        std::array<std::uint64_t, 2> values{
            generator_(),
            generator_()
        };

        std::ostringstream output;
        output << std::hex << std::setfill('0')
            << std::setw(16) << values[^1_0]
            << std::setw(16) << values[^1_1];
        return output.str();
    }

    std::mt19937_64 generator_;

    std::unordered_map<
        Token,
        Player*,

```

```

        util::TaggedHasher<Token>> token_to_player_;
    };

    } // namespace model

```

Почему токен формируется именно так

`std::uint64_t` в шестнадцатеричном виде занимает до 16 символов. Два числа по 64 бита дают ровно 32 hex-символа:

```

std::setw(16) << first // 16 hex-символов
std::setw(16) << second // ещё 16

```

Таким образом, результат удовлетворяет контракту:

```

6516861d89ebffff147bf2eb2b5153ae1
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ 32 символа [0-9a-f]

```

Не применяйте `std::to_string` — он формирует десятичную строку и нарушит требование.

Класс приложения

Создайте, например, `application.h`.

```

#pragma once

#include <memory>
#include <string>
#include <unordered_map>

#include "model.h"
#include "player.h"

namespace app {

class Application {
public:
    explicit Application(model::Game& game)
        : game_(game) {}

    struct JoinResult {
        model::PlayerId player_id;
        model::Token token;
    };

    JoinResult JoinGame(
        const model::Map::Id& map_id,
        std::string user_name) {

```

```

        const model::Map* map = game_.FindMap(map_id);
        if (map == nullptr) {
            throw std::out_of_range("Map not found");
        }

        auto& session = GetOrCreateSession(*map);
        model::Dog& dog = session.AddDog(std::move(user_name));
        model::Player& player = players_.Add(dog, session);
        model::Token token = tokens_.AddPlayer(player);

        return {player.GetId(), std::move(token)};
    }

    model::Player* FindPlayerByToken(
        const model::Token& token) const noexcept {
        return tokens_.FindPlayerByToken(token);
    }

private:
    model::GameSession& GetOrCreateSession(
        const model::Map& map) {

        const std::string map_id = *map.GetId();

        if (auto it = sessions_.find(map_id); it != sessions_.end()) {
            return *it->second;
        }

        auto session = std::make_unique<model::GameSession>(&map);
        auto* result = session.get();
        sessions_.emplace(map_id, std::move(session));
        return *result;
    }

    model::Game& game_;
    model::Players players_;
    model::PlayerTokens tokens_;
    std::unordered_map<
        std::string,
        std::unique_ptr<model::GameSession>> sessions_;
};

} // namespace app

```

Game уже умеет искать карту по Map::Id через FindMap, поэтому не нужно дублировать индекс карт в прикладном слое.^[1]

HTTP-обработчик

Добавьте зависимость и поля

В `request_handler.h`:

```
#include "application.h"
```

В конструктор передавайте `model::Game&`, а не храните копию:

```
explicit RequestHandler(
    model::Game& game,
    fs::path static_root,
    Strand api_strand)
    : game_(game)
    , app_(game_)
    , static_root_{
        fs::weakly_canonical(fs::absolute(std::move(static_root)))}
    , api_strand_(std::move(api_strand)) {
}
```

Поля класса:

```
model::Game& game_;
app::Application app_;
fs::path static_root_;
Strand api_strand_;
```

В `main.cpp` объект `game` уже живёт до завершения `ios.run()`, поэтому ссылку на него хранить корректно.^[4]

Общие JSON-ответы

Измените `MakeJsonResponse`, чтобы во всех API-ответах автоматически был `Cache-Control: no-cache`:

```
static StringResponse MakeJsonResponse(
    http::status status,
    unsigned version,
    bool keep_alive,
    json::value body) {

    StringResponse response{status, version};
    response.set(http::field::content_type, "application/json");
    response.set(http::field::cache_control, "no-cache");
    response.keep_alive(keep_alive);
    response.body() = json::serialize(body);
    response.prepare_payload();
    return response;
}
```

`prepare_payload()` автоматически выставит `Content-Length` согласно сериализованному телу. В текущем коде этот метод уже используется, поэтому отдельное ручное вычисление длины не требуется.^[3]

Проверка Content-Type

Добавьте помощник:

```
static bool IsJsonContentType(const StringRequest& request) {
    const auto content_type = request[http::field::content_type];

    const auto semicolon = content_type.find(';');
    const auto mime_type = content_type.substr(0, semicolon);

    return beast::iequals(mime_type, "application/json");
}
```

Это допускает распространённый допустимый вариант:

```
Content-Type: application/json; charset=utf-8
```

POST /api/v1/game/join

```
StringResponse HandleJoinGameRequest(
    const StringRequest& request) {

    if (request.method() != http::verb::post) {
        auto response = MakeErrorResponse(
            http::status::method_not_allowed,
            request.version(),
            request.keep_alive(),
            "invalidMethod",
            "Only POST method is expected");

        response.set(http::field::allow, "POST");
        return response;
    }

    if (!IsJsonContentType(request)) {
        return MakeErrorResponse(
            http::status::bad_request,
            request.version(),
            request.keep_alive(),
            "invalidArgument",
            "Invalid content type");
    }

    try {
        const json::value request_json = json::parse(request.body());
        const json::object& object = request_json.as_object();
    }
```

```

const std::string user_name =
    json::value_to<std::string>(object.at("userName"));

const std::string map_id =
    json::value_to<std::string>(object.at("mapId"));

if (user_name.empty()) {
    return MakeErrorResponse(
        http::status::bad_request,
        request.version(),
        request.keep_alive(),
        "invalidArgument",
        "Invalid name");
}

try {
    const auto result = app_.JoinGame(
        model::Map::Id{map_id},
        user_name);

    return MakeJsonResponse(
        http::status::ok,
        request.version(),
        request.keep_alive(),
        json::object{
            {"authToken", *result.token},
            {"playerId", *result.player_id},
        });
} catch (const std::out_of_range&) {
    return MakeErrorResponse(
        http::status::not_found,
        request.version(),
        request.keep_alive(),
        "mapNotFound",
        "Map not found");
}
} catch (...) {
    return MakeErrorResponse(
        http::status::bad_request,
        request.version(),
        request.keep_alive(),
        "invalidArgument",
        "Join game request parse error");
}
}

```

Важно: `json::parse`, `as_object`, `object.at`, `value_to<std::string>` могут бросать исключения при неверном JSON, отсутствии полей, неправильном типе поля и т. п. Все эти случаи должны быть сведены к единому ответу 400 `invalidArgument` / `Join game request parse error`.

Извлечение Bearer-токена

```
static std::optional<model::Token> ExtractBearerToken(
    const StringRequest& request) {

    const auto authorization = request[http::field::authorization];

    constexpr std::string_view prefix = "Bearer ";

    if (authorization.size() <= prefix.size()) {
        return std::nullopt;
    }

    const std::string_view value{
        authorization.data(),
        authorization.size()
    };

    if (!value.starts_with(prefix)) {
        return std::nullopt;
    }

    const std::string_view token = value.substr(prefix.size());

    if (token.size() != 32) {
        return std::nullopt;
    }

    for (const unsigned char ch : token) {
        if (!std::isxdigit(ch)) {
            return std::nullopt;
        }
    }

    return model::Token{std::string(token)};
}
```

Подключите:

```
#include <cctype>
#include <optional>
```

Разделяйте два случая:

Ситуация	Код	HTTP
Нет Authorization, неправильная схема, пустой/невалидный формат	invalidToken	401
Формат корректный, но сервер не знает токен	unknownToken	401
Токен есть и известен серверу	—	200

GET /api/v1/game/players

```
StringResponse HandlePlayersRequest(
    const StringRequest& request) {

    if (request.method() != http::verb::get &&
        request.method() != http::verb::head) {

        auto response = MakeErrorResponse(
            http::status::method_not_allowed,
            request.version(),
            request.keep_alive(),
            "invalidMethod",
            "Invalid method");

        response.set(http::field::allow, "GET, HEAD");
        return response;
    }

    const auto token = ExtractBearerToken(request);
    if (!token) {
        return MakeErrorResponse(
            http::status::unauthorized,
            request.version(),
            request.keep_alive(),
            "invalidToken",
            "Authorization header is missing");
    }

    const model::Player* current_player =
        app_.FindPlayerByToken(*token);

    if (current_player == nullptr) {
        return MakeErrorResponse(
            http::status::unauthorized,
            request.version(),
            request.keep_alive(),
            "unknownToken",
            "Player token has not been found");
    }

    json::object players_json;

    const auto& current_session =
        current_player->GetSession();

    for (const auto& [player_id, player] : app_.GetPlayers()) {
        if (&player.GetSession() != &current_session) {
            continue;
        }

        players_json.emplace(
            std::to_string(*player_id),
            json::object{
                {"name", player.GetDog().GetName()}
            });
    }
}
```

```

    }

    if (request.method() == http::verb::head) {
        auto response = MakeJsonResponse(
            http::status::ok,
            request.version(),
            request.keep_alive(),
            std::move(players_json));

        response.body().clear();
        response.content_length(0);
        return response;
    }

    return MakeJsonResponse(
        http::status::ok,
        request.version(),
        request.keep_alive(),
        std::move(players_json));
}

```

Для этого добавьте в Application доступ к игрокам:

```

const model::Players& GetPlayers() const noexcept {
    return players_;
}

```

А в Players удобный итераторный интерфейс:

```

auto begin() const noexcept {
    return players_.begin();
}

auto end() const noexcept {
    return players_.end();
}

```

Тогда цикл можно сделать проще:

```

for (const auto& [player_id, player] : app_.GetPlayers()) {
    // ...
}

```

Маршрутизация API

В месте, где сейчас определяется API-маршрут, добавьте константы:

```

constexpr std::string_view kJoinGameTarget =
    "/api/v1/game/join";

```

```
constexpr std::string_view kPlayersTarget =  
    "/api/v1/game/players";
```

В `HandleApiRequest`:

```
StringResponse HandleApiRequest(  
    const StringRequest& request) const {  
  
    const std::string_view target = request.target();  
  
    if (target == "/api/v1/game/join") {  
        return const_cast<RequestHandler*>(this)  
            ->HandleJoinGameRequest(request);  
    }  
  
    if (target == "/api/v1/game/players") {  
        return const_cast<RequestHandler*>(this)  
            ->HandlePlayersRequest(request);  
    }  
  
    // Здесь остаются существующие /api/v1/maps и /api/v1/maps/{id}.  
    return MakeErrorResponse(  
        http::status::bad_request,  
        request.version(),  
        request.keep_alive(),  
        "badRequest",  
        "Bad request");  
}
```

Лучше сразу убрать `const` у `HandleApiRequest`, потому что `join` изменяет состояние `app_`:

```
StringResponse HandleApiRequest(const StringRequest& request);
```

И тогда `const_cast` не понадобится.

Что поправить в `main.cpp`

Создайте API strand и передайте его обработчику:

```
auto api_strand = net::make_strand(ioc);  
  
http_handler::RequestHandler handler{  
    game,  
    static_root,  
    std::move(api_strand)  
};
```

Это соответствует вашему текущему шаблону `RequestHandler`, где API-ветка должна быть выполнена через `api_strand_`.^[3]

Проверка через curl

Успешный вход

```
curl -i \  
  -X POST http://localhost:8080/api/v1/game/join \  
  -H 'Content-Type: application/json' \  
  -d '{"userName":"Scooby Doo","mapId":"map1"}'
```

Ожидаемая структура:

```
HTTP/1.1 200 OK  
Content-Type: application/json  
Cache-Control: no-cache  
  
{"authToken":"<32 hex chars>","playerId":0}
```

Игроки на карте

```
curl -i \  
  http://localhost:8080/api/v1/game/players \  
  -H 'Authorization: Bearer <TOKEN_ИЗ_JOIN>'
```

Ожидаемый ответ:

```
{  
  "0": {  
    "name": "Scooby Doo"  
  }  
}
```

Неверная карта

```
curl -i \  
  -X POST http://localhost:8080/api/v1/game/join \  
  -H 'Content-Type: application/json' \  
  -d '{"userName":"Scooby Doo","mapId":"does-not-exist"}'
```

Должно быть:

```
{  
  "code": "mapNotFound",  
  "message": "Map not found"  
}
```

Некорректный JSON

```
curl -i \
  -X POST http://localhost:8080/api/v1/game/join \
  -H 'Content-Type: application/json' \
  -d '{"userName":'
```

Должно быть:

```
{
  "code": "invalidArgument",
  "message": "Join game request parse error"
}
```

Важные замечания

- Не храните ``Dog*`` в ``std::vector<Dog>``, если в будущем вектор может перераспределить

- Не храните игроков в `std::vector<Player>`, если `PlayerTokens` содержит `Player*`: при расширении вектора адреса игроков могут измениться. `std::unordered_map<PlayerId, Player>` подходит лучше, поскольку ссылки и указатели на его элементы не инвалидируются при `rehash`.
- `Cache-Control: no-cache` должен быть и у успешных, и у ошибочных API-ответов. Централизованная установка в `MakeJsonResponse` предотвращает пропуски.
- Статические файлы не должны обрабатываться в API strand: они могут обслуживаться параллельно. Последовательной должна быть только игровая изменяемая модель. Такая граница у вас уже просматривается в `RequestHandler`.^[3]
- `Content-Length` не стоит задавать руками для JSON: `response.prepare_payload()` корректно рассчитает его по `body()`.^[3]

✱

результатирующий код - прошу проверить

Код в текущем виде не соберётся и не сможет обработать новые endpoint'ы. Основная причина: часть реализации добавлена в `main.cpp`, часть — в `request_handler.cpp`, но методы не являются методами `RequestHandler`, а маршрутизация API всё ещё не реализована. Ниже — конкретные проблемы в порядке критичности и точные исправления.^{[15] [16] [17] [18] [19]}

Критические ошибки сборки

1. application.h: неполные шаблонные аргументы

В GetOrCreateSession сейчас:

```
auto session = std::make_unique(&map);
```

Так не скомпилируется: компилятор не может вывести тип для std::make_unique.

Должно быть:

```
auto session = std::make_unique<model::GameSession>(&map);
```

Поле sessions_ тоже записано без типа аргумента unique_ptr:

```
std::unordered_map<
    std::string,
    std::unique_ptr> sessions_;
```

Нужно:

```
std::unordered_map<
    std::string,
    std::unique_ptr<model::GameSession>> sessions_;
```

И обязательно нужны заголовки:

```
#include <memory>
#include <stdexcept>
#include <string>
#include <unordered_map>
#include <utility>
```

Итоговый исправленный фрагмент:

```
private:
    model::GameSession& GetOrCreateSession(const model::Map& map) {
        const std::string map_id = *map.GetId();

        if (auto it = sessions_.find(map_id); it != sessions_.end()) {
            return *it->second;
        }

        auto session = std::make_unique<model::GameSession>(&map);
        model::GameSession* result = session.get();

        sessions_.emplace(map_id, std::move(session));
    }
```

```

        return *result;
    }

    model::Game& game_;
    model::Players players_;
    model::PlayerTokens tokens_;

    std::unordered_map<
        std::string,
        std::unique_ptr<model::GameSession>> sessions_;

```

2. В `main.cpp` находятся методы, которые там не могут работать

В начале `main.cpp` добавлены:

```
StringResponse HandleJoinGameRequest(...)
```

и:

```
StringResponse HandlePlayersRequest(...)
```

Но в этом файле нет доступных имён:

- `StringResponse`;
- `StringRequest`;
- `http`;
- `json`;
- `app_`;
- `MakeErrorResponse`;
- `MakeJsonResponse`;
- `IsJsonContentType`.

Все они принадлежат `http_handler::RequestHandler` или должны быть определены в его `namespace/class`. Следовательно, эти функции в `main.cpp` не соберутся. ^[16] ^[18]

Исправление: полностью удалите `HandleJoinGameRequest`, `ExtractBearerToken` и `HandlePlayersRequest` из `main.cpp`.

В `main.cpp` должны остаться только:

- загрузка конфигурации;
- создание `Game`;
- создание `io_context`;
- создание `api_strand`;
- создание `RequestHandler`;

- запуск HTTP-сервера и worker-потоков.

3. request_handler.cpp: функция определена не как метод класса

В заголовке объявлено:

```
StringResponse HandleJoinGameRequest(const StringRequest& request);
```

Это **приватный метод** http_handler::RequestHandler. Но в request_handler.cpp определено:

```
StringResponse HandleJoinGameRequest(  
    const StringRequest& request) {
```

Это свободная функция в namespace http_handler, а не метод класса.

Нужно:

```
http_handler::RequestHandler::StringResponse  
http_handler::RequestHandler::HandleJoinGameRequest(  
    const StringRequest& request) {  
    // ...  
}
```

Однако StringResponse — приватный alias. Поэтому более аккуратный вариант внутри namespace:

```
RequestHandler::StringResponse RequestHandler::HandleJoinGameRequest(  
    const StringRequest& request) {  
    // ...  
}
```

Но у вас также StringRequest, MakeErrorResponse, MakeJsonResponse, IsJsonContentType являются членами класса. При правильном определении метода они будут доступны.

Текущий request_handler.cpp не соберётся, потому что свободная функция не видит:

```
StringResponse  
StringRequest  
MakeErrorResponse  
IsJsonContentType  
app_  
MakeJsonResponse
```

4. HandleApiRequest **объявлен const, но join меняет состояние**

В request_handler.h:

```
StringResponse HandleApiRequest(const StringRequest& request) const;
```

Но join вызывает:

```
app_.JoinGame(...)
```

А JoinGame меняет сессии, игроков и токены. Значит HandleApiRequest не должен быть const.

Исправьте объявление:

```
StringResponse HandleApiRequest(const StringRequest& request);
```

Это важно и для логики, и для компиляции при вызове HandleJoinGameRequest.

[\[18\]](#) [\[15\]](#)

5. **Объявлен только** HandleJoinGameRequest

В классе есть:

```
StringResponse HandleJoinGameRequest(const StringRequest& request);
```

Но отсутствует:

```
StringResponse HandlePlayersRequest(const StringRequest& request);
```

Добавьте:

```
StringResponse HandlePlayersRequest(const StringRequest& request);
```

ExtractBearerToken лучше сделать static приватным методом RequestHandler:

```
static std::optional<model::Token> ExtractBearerToken(  
    const StringRequest& request);
```

И добавьте include:

```
#include <optional>
```

Ошибки модели и времени жизни

6. Dog* станет висячим после перераспределения std::vector

В GameSession собаки хранятся в:

```
std::vector<Dog> dogs_;
```

Player сохраняет указатель:

```
Dog* dog_;
```

После добавления очередной собаки std::vector может перераспределить память. Тогда адреса всех старых Dog изменятся, а ранее созданные Player::dog_ станут висячими указателями. /players в таком случае может упасть или вернуть повреждённые данные.

Нужно заменить:

```
std::vector<Dog> dogs_;
```

на:

```
std::deque<Dog> dogs_;
```

И:

```
const std::vector<Dog>& GetDogs() const noexcept {  
    return dogs_;  
}
```

на:

```
const std::deque<Dog>& GetDogs() const noexcept {  
    return dogs_;  
}
```

Добавьте:

```
#include <deque>
```

std::deque сохраняет адреса уже существующих элементов при emplace_back, что делает Dog* в Player стабильным. ^[17]

7. Player* в PlayerTokens хранится корректно

Это место реализовано удачно:

```
std::unordered_map<PlayerId, Player, util::TaggedHasher<PlayerId>> players_;
```

и:

```
std::unordered_map<Token, Player*, util::TaggedHasher<Token>> token_to_player_;
```

Ссылки и указатели на элементы `std::unordered_map` не инвалидируются при `rehash`. То есть `Player*`, сохранённый в `PlayerTokens`, остаётся валидным при добавлении других игроков. Это хорошее решение. ^[17]

8. Токен формируется корректно

Реализация:

```
std::array values{
    generator_(),
    generator_()
};

output << std::hex << std::setfill('0')
        << std::setw(16) << values[^2_0]
        << std::setw(16) << values[^2_1];
```

действительно генерирует строку из 32 шестнадцатеричных символов. `std::setw(16)` гарантирует ведущие нули, поэтому длина будет ровно 32 символа. Это соответствует заданию. ^[17]

Но для корректной сборки лучше сделать тип массива явным:

```
std::array<std::uint64_t, 2> values{
    generator_(),
    generator_()
};
```

Текущий вариант с CTAD в C20 тоже допустим, если проект собирается именно с C20.

Ошибки API и маршрутизации

9. API-маршрутизация до сих пор не реализована

В `RequestHandler::operator()` всё ещё осталось:

```
if (/*req относится к API?*/) {
```

Это нерабочая заглушка. Пока она не заменена настоящей проверкой, сервер не скомпилируется.

Добавьте:

```
static bool IsApiRequest(const StringRequest& request) {
    constexpr std::string_view kApiPrefix = "/api/";
    const std::string_view target{
        request.target().data(),
        request.target().size()
    };
    return target.starts_with(kApiPrefix);
}
```

И используйте:

```
if (IsApiRequest(req)) {
```

Либо без helper-функции:

```
const std::string_view target{
    req.target().data(),
    req.target().size()
};

if (target.starts_with("/api/")) {
    // dispatch to api_strand_
}
```

Без этого игровая логика не попадёт в `api_strand_`, а запросы `/api/v1/game/join` и `/api/v1/game/players` не будут обработаны. ^[18]

10. `HandleApiRequest` должен маршрутизировать новые endpoint'ы

Нужно реализовать или расширить метод в `request_handler.cpp`:

```
RequestHandler::StringResponse RequestHandler::HandleApiRequest(
    const StringRequest& request) {

    const std::string_view target{
        request.target().data(),
        request.target().size()
    };
}
```

```

if (target == "/api/v1/game/join") {
    return HandleJoinGameRequest(request);
}

if (target == "/api/v1/game/players") {
    return HandlePlayersRequest(request);
}

if (target == "/api/v1/maps") {
    if (request.method() != http::verb::get) {
        return MakeErrorResponse(
            http::status::bad_request,
            request.version(),
            request.keep_alive(),
            "badRequest",
            "Bad request");
    }

    return MakeMapsResponse(
        request.version(),
        request.keep_alive());
}

constexpr std::string_view kMapsPrefix = "/api/v1/maps/";

if (target.starts_with(kMapsPrefix)) {
    if (request.method() != http::verb::get) {
        return MakeErrorResponse(
            http::status::bad_request,
            request.version(),
            request.keep_alive(),
            "badRequest",
            "Bad request");
    }

    const std::string_view map_id = target.substr(kMapsPrefix.size());

    if (map_id.empty() || map_id.find('/') != std::string_view::npos) {
        return MakeErrorResponse(
            http::status::bad_request,
            request.version(),
            request.keep_alive(),
            "badRequest",
            "Bad request");
    }

    const model::Map* map = game_.FindMap(
        model::Map::Id{std::string(map_id)});

    if (map == nullptr) {
        return MakeErrorResponse(
            http::status::not_found,
            request.version(),
            request.keep_alive(),
            "mapNotFound",

```

```

        "Map not found");
    }

    return MakeMapResponse(
        *map,
        request.version(),
        request.keep_alive());
}

return MakeErrorResponse(
    http::status::bad_request,
    request.version(),
    request.keep_alive(),
    "badRequest",
    "Bad request");
}

```

Сейчас `HandleApiRequest` только объявлен, но в предоставленном коде его определения нет. [\[19\]](#) [\[18\]](#)

11. `HandlePlayersRequest` **не должен быть** в `main.cpp`

Переместите его в `request_handler.cpp` и определите как:

```

RequestHandler::StringResponse RequestHandler::HandlePlayersRequest(
    const StringRequest& request) {
    // реализация
}

```

Там же поместите:

```

std::optional<model::Token> RequestHandler::ExtractBearerToken(
    const StringRequest& request) {
    // реализация
}

```

Это обеспечит доступ к `app_`, `MakeErrorResponse`, `MakeJsonResponse`, `http` и `json`.

HEAD для `/players`

В задании допустимы GET и HEAD. Ваше намерение поддержать HEAD правильное, но реализация имеет неприятный нюанс:

```

auto response = MakeJsonResponse(..., std::move(players_json));
response.body().clear();
response.content_length(0);
return response;

```

HEAD обычно должен вернуть те же заголовки, что и соответствующий GET, включая **Content-Length** размера GET-тела, но без самого тела. В учебных тестах могут проверять

только статус и отсутствие body, однако более корректный вариант:

```
if (request.method() == http::verb::head) {
    const std::string body = json::serialize(players_json);

    http::response<http::empty_body> response{
        http::status::ok,
        request.version()
    };

    response.set(http::field::content_type, "application/json");
    response.set(http::field::cache_control, "no-cache");
    response.content_length(body.size());
    response.keep_alive(request.keep_alive());
    return response; // Потребуется отдельный тип результата.
}
```

Но это усложняет ваш единый тип `StringResponse`. Поэтому для текущего этапа допустимо вернуть пустое тело с `Content-Length: 0`, если тесты не требуют RFC-поведения HEAD. Более важно сначала переместить обработчик в `RequestHandler` и подключить маршрутизацию. ^[16] ^[18]

Исправленный `request_handler.cpp`

Ниже рабочий каркас именно для новых endpoint'ов. Он предполагает, что в `request_handler.h` добавлены объявления из следующего раздела.

```
#include "request_handler.h"

#include <cctype>
#include <optional>
#include <string_view>

namespace http_handler {

std::optional<model::Token> RequestHandler::ExtractBearerToken(
    const StringRequest& request) {

    constexpr std::string_view kPrefix = "Bearer ";

    const auto authorization = request[http::field::authorization];

    const std::string_view value{
        authorization.data(),
        authorization.size()
    };

    if (!value.starts_with(kPrefix)) {
        return std::nullopt;
    }

    const std::string_view token = value.substr(kPrefix.size());
```

```

    if (token.size() != 32) {
        return std::nullopt;
    }

    for (const unsigned char ch : token) {
        if (!std::isxdigit(ch)) {
            return std::nullopt;
        }
    }

    return model::Token{std::string(token)};
}

RequestHandler::StringResponse RequestHandler::HandleJoinGameRequest(
    const StringRequest& request) {

    if (request.method() != http::verb::post) {
        auto response = MakeErrorResponse(
            http::status::method_not_allowed,
            request.version(),
            request.keep_alive(),
            "invalidMethod",
            "Only POST method is expected");

        response.set(http::field::allow, "POST");
        return response;
    }

    if (!IsJsonContentType(request)) {
        return MakeErrorResponse(
            http::status::bad_request,
            request.version(),
            request.keep_alive(),
            "invalidArgument",
            "Invalid content type");
    }

    try {
        const json::value request_json = json::parse(request.body());
        const json::object& object = request_json.as_object();

        const std::string user_name =
            json::value_to<std::string>(object.at("userName"));

        const std::string map_id =
            json::value_to<std::string>(object.at("mapId"));

        if (user_name.empty()) {
            return MakeErrorResponse(
                http::status::bad_request,
                request.version(),
                request.keep_alive(),
                "invalidArgument",
                "Invalid name");
        }
    }

```

```

    try {
        const auto result = app_.JoinGame(
            model::Map::Id{map_id},
            user_name);

        return MakeJsonResponse(
            http::status::ok,
            request.version(),
            request.keep_alive(),
            json::object{
                {"authToken", *result.token},
                {"playerId", *result.player_id},
            });
    } catch (const std::out_of_range&) {
        return MakeErrorResponse(
            http::status::not_found,
            request.version(),
            request.keep_alive(),
            "mapNotFound",
            "Map not found");
    }
} catch (...) {
    return MakeErrorResponse(
        http::status::bad_request,
        request.version(),
        request.keep_alive(),
        "invalidArgument",
        "Join game request parse error");
}
}

RequestHandler::StringResponse RequestHandler::HandlePlayersRequest(
    const StringRequest& request) {

    if (request.method() != http::verb::get &&
        request.method() != http::verb::head) {

        auto response = MakeErrorResponse(
            http::status::method_not_allowed,
            request.version(),
            request.keep_alive(),
            "invalidMethod",
            "Invalid method");

        response.set(http::field::allow, "GET, HEAD");
        return response;
    }

    const auto token = ExtractBearerToken(request);

    if (!token) {
        return MakeErrorResponse(
            http::status::unauthorized,
            request.version(),
            request.keep_alive(),
            "invalidToken",

```

```

        "Authorization header is missing");
    }

    const model::Player* current_player =
        app_.FindPlayerByToken(*token);

    if (current_player == nullptr) {
        return MakeErrorResponse(
            http::status::unauthorized,
            request.version(),
            request.keep_alive(),
            "unknownToken",
            "Player token has not been found");
    }

    const model::GameSession& current_session =
        current_player->GetSession();

    json::object players_json;

    for (const auto& [player_id, player] : app_.GetPlayers()) {
        if (&player.GetSession() != &current_session) {
            continue;
        }

        players_json.emplace(
            std::to_string(*player_id),
            json::object{
                {"name", player.GetDog().GetName()}
            });
    }

    auto response = MakeJsonResponse(
        http::status::ok,
        request.version(),
        request.keep_alive(),
        std::move(players_json));

    if (request.method() == http::verb::head) {
        response.body().clear();
        response.content_length(0);
    }

    return response;
}

} // namespace http_handler

```

Изменения в request_handler.h

В private-секции должны быть такие объявления:

```

using StringRequest = http::request<http::string_body>;
using StringResponse = http::response<http::string_body>;

```

```
FileRequestResult HandleFileRequest(const StringRequest& req) const;

StringResponse HandleApiRequest(const StringRequest& request);

StringResponse HandleJoinGameRequest(
    const StringRequest& request);

StringResponse HandlePlayersRequest(
    const StringRequest& request);

StringResponse ReportServerError(
    unsigned version,
    bool keep_alive) const;

static std::optional<model::Token> ExtractBearerToken(
    const StringRequest& request);
```

И обновите API-признак:

```
static bool IsApiRequest(const StringRequest& request) {
    constexpr std::string_view kApiPrefix = "/api/";

    const std::string_view target{
        request.target().data(),
        request.target().size()
    };

    return target.starts_with(kApiPrefix);
}
```

В оператор() замените заглушку:

```
if (/*req относится к API?*/) {
```

на:

```
if (IsApiRequest(req)) {
```

Также убедитесь, что HandleApiRequest объявлен без const. ^[18]

Обязательная правка player.h

Замените контейнер собак на deque:

```
#include <deque>
```

```
class GameSession {
public:
    explicit GameSession(const Map* map)
```

```

        : map_(map) {
    }

    const Map& GetMap() const noexcept {
        return *map_;
    }

    Dog& AddDog(std::string name) {
        const DogId id{next_dog_id_++};
        return dogs_.emplace_back(id, std::move(name));
    }

    const std::deque<Dog>& GetDogs() const noexcept {
        return dogs_;
    }

private:
    const Map* map_;
    std::uint64_t next_dog_id_ = 0;
    std::deque<Dog> dogs_;
};

```

Это устранил повреждение `Player::dog_` после нескольких `POST /join`.^[17]

Минимальный чек-лист

После исправлений проверьте именно эти сценарии:

```

# 1. Успешный join
curl -i -X POST http://localhost:8080/api/v1/game/join \
  -H 'Content-Type: application/json' \
  -d '{"userName":"Scooby Doo","mapId":"map1"}'

# 2. Второй игрок на той же карте
curl -i -X POST http://localhost:8080/api/v1/game/join \
  -H 'Content-Type: application/json' \
  -d '{"userName":"Shaggy","mapId":"map1"}'

# 3. Игроки той же сессии
curl -i http://localhost:8080/api/v1/game/players \
  -H 'Authorization: Bearer <TOKEN>'

# 4. Неверный метод join
curl -i http://localhost:8080/api/v1/game/join

# 5. Неизвестный токен
curl -i http://localhost:8080/api/v1/game/players \
  -H 'Authorization: Bearer 00000000000000000000000000000000'

# 6. Нет Authorization
curl -i http://localhost:8080/api/v1/game/players

```

В текущем присланном результате сначала исправьте: ``application.h``, перенос функций

выведи результирующие файлы, которые надо поменять

Нужно заменить или добавить пять файлов:

1. `player.h` — заменить полностью.
2. `application.h` — заменить полностью.
3. `request_handler.h` — заменить полностью.
4. `request_handler.cpp` — заменить полностью.
5. `main.cpp` — заменить полностью.

`model.h`, `model.cpp`, `json_loader.*`, `http_server.*`, `tagged.h`, `sdk.h` менять не нужно.

Текущий `RequestHandler` уже содержит статическую раздачу файлов и сериализацию карт; ниже эти части сохранены, а API расширен `/api/v1/game/join` и `/api/v1/game/players`. [\[31\]](#) [\[32\]](#) [\[33\]](#) [\[34\]](#) [\[35\]](#)

`player.h`

```
#pragma once

#include <array>
#include <cstdint>
#include <deque>
#include <iomanip>
#include <random>
#include <sstream>
#include <string>
#include <unordered_map>
#include <utility>

#include "model.h"
#include "tagged.h"

namespace model {

namespace detail {

struct DogTag {};
struct PlayerTag {};
struct TokenTag {};

} // namespace detail

using DogId = util::Tagged<std::uint64_t, detail::DogTag>;
using PlayerId = util::Tagged<std::uint64_t, detail::PlayerTag>;
using Token = util::Tagged<std::string, detail::TokenTag>;

class Dog {
```

```

public:
    Dog(DogId id, std::string name)
        : id_(id)
        , name_(std::move(name)) {
    }

    DogId GetId() const noexcept {
        return id_;
    }

    const std::string& GetName() const noexcept {
        return name_;
    }

private:
    DogId id_;
    std::string name_;
};

class GameSession {
public:
    explicit GameSession(const Map* map)
        : map_(map) {
    }

    const Map& GetMap() const noexcept {
        return *map_;
    }

    Dog& AddDog(std::string name) {
        const DogId id{next_dog_id_++};
        return dogs_.emplace_back(id, std::move(name));
    }

    const std::deque<Dog>& GetDogs() const noexcept {
        return dogs_;
    }

private:
    const Map* map_;
    std::uint64_t next_dog_id_ = 0;

    // deque нужен, чтобы адреса Dog не менялись при добавлении новых собак.
    // Player хранит Dog*, поэтому vector здесь использовать опасно.
    std::deque<Dog> dogs_;
};

class Player {
public:
    Player(PlayerId id, GameSession& session, Dog& dog) noexcept
        : id_(id)
        , session_(&session)
        , dog_(&dog) {
    }

    PlayerId GetId() const noexcept {

```

```

        return id_;
    }

    const GameSession& GetSession() const noexcept {
        return *session_;
    }

    const Dog& GetDog() const noexcept {
        return *dog_;
    }

private:
    PlayerId id_;
    GameSession* session_;
    Dog* dog_;
};

class Players {
public:
    Player& Add(Dog& dog, GameSession& session) {
        const PlayerId id{next_player_id_++};

        auto [it, inserted] = players_.emplace(
            id,
            Player{id, session, dog});

        return it->second;
    }

    Player* FindById(PlayerId id) noexcept {
        if (auto it = players_.find(id); it != players_.end()) {
            return &it->second;
        }
        return nullptr;
    }

    auto begin() const noexcept {
        return players_.begin();
    }

    auto end() const noexcept {
        return players_.end();
    }

private:
    std::uint64_t next_player_id_ = 0;

    // Указатели на элементы unordered_map не инвалидируются при rehash.
    // Это важно, поскольку PlayerTokens хранит Player*.
    std::unordered_map<
        PlayerId,
        Player,
        util::TaggedHasher<PlayerId>> players_;
};

class PlayerTokens {

```

```

public:
    PlayerTokens()
        : generator_(std::random_device{}()) {
    }

    Token AddPlayer(Player& player) {
        Token token{std::string{}};

        do {
            token = Token{GenerateToken()};
        } while (token_to_player_.contains(token));

        token_to_player_.emplace(token, &player);
        return token;
    }

    Player* FindPlayerByToken(const Token& token) const noexcept {
        if (auto it = token_to_player_.find(token);
            it != token_to_player_.end()) {
            return it->second;
        }

        return nullptr;
    }

private:
    std::string GenerateToken() {
        const std::array<std::uint64_t, 2> values{
            generator_(),
            generator_()
        };

        std::ostringstream output;
        output << std::hex << std::setfill('0')
            << std::setw(16) << values[^3_0]
            << std::setw(16) << values[^3_1];

        return output.str();
    }

    std::mt19937_64 generator_;

    std::unordered_map<
        Token,
        Player*,
        util::TaggedHasher<Token>> token_to_player_
    >;

} // namespace model

```

application.h

```
#pragma once

#include <memory>
#include <stdexcept>
#include <string>
#include <unordered_map>
#include <utility>

#include "model.h"
#include "player.h"

namespace app {

class Application {
public:
    explicit Application(model::Game& game)
        : game_(game) {}

    struct JoinResult {
        model::PlayerId player_id;
        model::Token token;
    };

    JoinResult JoinGame(
        const model::Map::Id& map_id,
        std::string user_name) {

        const model::Map* map = game_.FindMap(map_id);

        if (map == nullptr) {
            throw std::out_of_range("Map not found");
        }

        model::GameSession& session = GetOrCreateSession(*map);
        model::Dog& dog = session.AddDog(std::move(user_name));
        model::Player& player = players_.Add(dog, session);
        model::Token token = tokens_.AddPlayer(player);

        return {player.GetId(), std::move(token)};
    }

    model::Player* FindPlayerByToken(
        const model::Token& token) const noexcept {

        return tokens_.FindPlayerByToken(token);
    }

    const model::Players& GetPlayers() const noexcept {
        return players_;
    }

private:
    model::GameSession& GetOrCreateSession(const model::Map& map) {
```

```

        const std::string map_id = *map.GetId();

        if (auto it = sessions_.find(map_id); it != sessions_.end()) {
            return *it->second;
        }

        auto session = std::make_unique<model::GameSession>(&map);
        model::GameSession* result = session.get();

        sessions_.emplace(map_id, std::move(session));
        return *result;
    }

    model::Game& game_;
    model::Players players_;
    model::PlayerTokens tokens_;

    std::unordered_map<
        std::string,
        std::unique_ptr<model::GameSession>> sessions_;
};

} // namespace app

```

request_handler.h

```

#pragma once

#include <algorithm>
#include <cassert>
#include <cctype>
#include <filesystem>
#include <optional>
#include <string>
#include <string_view>
#include <utility>
#include <variant>

#include <boost/asio/dispatch.hpp>
#include <boost/asio/strand.hpp>
#include <boost/beast/core.hpp>
#include <boost/beast/http.hpp>
#include <boost/json.hpp>

#include "application.h"
#include "model.h"

namespace http_handler {

namespace net = boost::asio;
namespace beast = boost::beast;
namespace http = beast::http;
namespace json = boost::json;
namespace fs = std::filesystem;

```

```

using tcp = net::ip::tcp;

class RequestHandler : public std::enable_shared_from_this<RequestHandler> {
public:
    using Strand = net::strand<net::io_context::executor_type>;

    explicit RequestHandler(
        model::Game& game,
        fs::path static_root,
        Strand api_strand)
        : game_(game)
        , app_(game_)
        , static_root_{
            fs::weakly_canonical(fs::absolute(std::move(static_root)))}
        , api_strand_(std::move(api_strand)) {
    }

    RequestHandler(const RequestHandler&) = delete;
    RequestHandler& operator=(const RequestHandler&) = delete;

    template <typename Body, typename Allocator, typename Send>
    void operator()(
        tcp::endpoint,
        http::request<Body, http::basic_fields<Allocator>>&& req,
        Send&& send) {

        const unsigned version = req.version();
        const bool keep_alive = req.keep_alive();

        try {
            if (IsApiRequest(req)) {
                auto handle = [
                    self = shared_from_this(),
                    req = std::forward<decltype(req)>(req),
                    send,
                    version,
                    keep_alive
               ]() mutable {
                    try {
                        assert(self->api_strand_.running_in_this_thread());

                        http::request<http::string_body> string_request{
                            std::move(req)};

                        send(self->HandleApiRequest(string_request));
                    } catch (...) {
                        send(self->ReportServerError(version, keep_alive));
                    }
                };

                net::dispatch(api_strand_, std::move(handle));
                return;
            }

            http::request<http::string_body> string_request{
                std::move(req)};

```

```

        std::visit(
            [&send](auto&& result) {
                send(std::forward<decltype(result)>(result));
            },
            HandleFileRequest(string_request));
    } catch (...) {
        send(ReportServerError(version, keep_alive));
    }
}

private:
    using StringRequest = http::request<http::string_body>;
    using StringResponse = http::response<http::string_body>;
    using EmptyResponse = http::response<http::empty_body>;
    using FileResponse = http::response<http::file_body>;

    using FileRequestResult = std::variant<
        StringResponse,
        EmptyResponse,
        FileResponse>;

    FileRequestResult HandleFileRequest(
        const StringRequest& req) const;

    StringResponse HandleApiRequest(
        const StringRequest& request);

    StringResponse HandleJoinGameRequest(
        const StringRequest& request);

    StringResponse HandlePlayersRequest(
        const StringRequest& request);

    StringResponse ReportServerError(
        unsigned version,
        bool keep_alive) const;

    model::Game& game_;
    app::Application app_;
    fs::path static_root_;
    Strand api_strand_;

    template <typename Body, typename Allocator>
    static bool IsApiRequest(
        const http::request<Body, http::basic_fields<Allocator>>& request) {

        constexpr std::string_view kApiPrefix = "/api/";

        const std::string_view target{
            request.target().data(),
            request.target().size()
        };

        return target.starts_with(kApiPrefix);
    }
}

```

```

static std::string ToLower(std::string value) {
    std::transform(
        value.begin(),
        value.end(),
        value.begin(),
        [](unsigned char ch) {
            return static_cast<char>(std::tolower(ch));
        });

    return value;
}

static std::string GetMimeType(const fs::path& path) {
    const std::string extension = ToLower(path.extension().string());

    if (extension == ".htm" || extension == ".html") {
        return "text/html";
    }
    if (extension == ".css") {
        return "text/css";
    }
    if (extension == ".txt") {
        return "text/plain";
    }
    if (extension == ".js") {
        return "text/javascript";
    }
    if (extension == ".json") {
        return "application/json";
    }
    if (extension == ".xml") {
        return "application/xml";
    }
    if (extension == ".png") {
        return "image/png";
    }
    if (extension == ".jpg" ||
        extension == ".jpeg" ||
        extension == ".jpe") {
        return "image/jpeg";
    }
    if (extension == ".gif") {
        return "image/gif";
    }
    if (extension == ".bmp") {
        return "image/bmp";
    }
    if (extension == ".ico") {
        return "image/vnd.microsoft.icon";
    }
    if (extension == ".tiff" || extension == ".tif") {
        return "image/tiff";
    }
    if (extension == ".svg" || extension == ".svgz") {
        return "image/svg+xml";
    }
}

```

```

    }
    if (extension == ".mp3") {
        return "audio/mpeg";
    }

    return "application/octet-stream";
}

static std::optional<std::string> UrlDecode(
    beast::string_view encoded) {

    const auto hex_to_int = [](char ch) -> int {
        if (ch >= '0' && ch <= '9') {
            return ch - '0';
        }
        if (ch >= 'a' && ch <= 'f') {
            return ch - 'a' + 10;
        }
        if (ch >= 'A' && ch <= 'F') {
            return ch - 'A' + 10;
        }
        return -1;
    };

    std::string decoded;
    decoded.reserve(encoded.size());

    for (size_t i = 0; i < encoded.size(); ++i) {
        const char ch = encoded[i];

        if (ch != '%') {
            decoded.push_back(ch);
            continue;
        }

        if (i + 2 >= encoded.size()) {
            return std::nullopt;
        }

        const int high = hex_to_int(encoded[i + 1]);
        const int low = hex_to_int(encoded[i + 2]);

        if (high < 0 || low < 0) {
            return std::nullopt;
        }

        decoded.push_back(
            static_cast<char>((high << 4) | low));

        i += 2;
    }

    return decoded;
}

static bool IsSubPath(

```

```

const fs::path& path,
const fs::path& base) {

const fs::path normalized_path = fs::weakly_canonical(path);
const fs::path normalized_base = fs::weakly_canonical(base);

auto path_it = normalized_path.begin();

for (auto base_it = normalized_base.begin();
     base_it != normalized_base.end();
     ++base_it, ++path_it) {
    if (path_it == normalized_path.end() ||
        *path_it != *base_it) {
        return false;
    }
}

return true;
}

static StringResponse MakeTextResponse(
    http::status status,
    unsigned version,
    bool keep_alive,
    std::string body) {

    StringResponse response{status, version};

    response.set(http::field::content_type, "text/plain");
    response.keep_alive(keep_alive);
    response.body() = std::move(body);
    response.prepare_payload();

    return response;
}

static StringResponse MakeJsonResponse(
    http::status status,
    unsigned version,
    bool keep_alive,
    json::value body) {

    StringResponse response{status, version};

    response.set(http::field::content_type, "application/json");
    response.set(http::field::cache_control, "no-cache");
    response.keep_alive(keep_alive);
    response.body() = json::serialize(body);
    response.prepare_payload();

    return response;
}

static StringResponse MakeErrorResponse(
    http::status status,
    unsigned version,

```

```

    bool keep_alive,
    std::string_view code,
    std::string_view message) {

    return MakeJsonResponse(
        status,
        version,
        keep_alive,
        json::object{
            {"code", code},
            {"message", message},
        });
}

static bool IsJsonContentType(
    const StringRequest& request) {

    const auto content_type = request[http::field::content_type];
    const size_t semicolon = content_type.find(';');
    const auto media_type = content_type.substr(0, semicolon);

    return beast::iequals(media_type, "application/json");
}

static std::optional<model::Token> ExtractBearerToken(
    const StringRequest& request);

static json::object SerializeRoad(const model::Road& road) {
    const model::Point start = road.GetStart();
    const model::Point end = road.GetEnd();

    if (road.IsHorizontal()) {
        return {
            {"x0", start.x},
            {"y0", start.y},
            {"x1", end.x},
        };
    }

    return {
        {"x0", start.x},
        {"y0", start.y},
        {"y1", end.y},
    };
}

static json::object SerializeBuilding(
    const model::Building& building) {

    const model::Rectangle& bounds = building.GetBounds();

    return {
        {"x", bounds.position.x},
        {"y", bounds.position.y},
        {"w", bounds.size.width},
        {"h", bounds.size.height},
    };
}

```

```

    };
}

static json::object SerializeOffice(
    const model::Office& office) {

    const model::Point position = office.GetPosition();
    const model::Offset offset = office.GetOffset();

    return {
        {"id", *office.GetId()},
        {"x", position.x},
        {"y", position.y},
        {"offsetX", offset.dx},
        {"offsetY", offset.dy},
    };
}

static json::object SerializeMap(const model::Map& map) {
    json::array roads;
    roads.reserve(map.GetRoads().size());

    for (const model::Road& road : map.GetRoads()) {
        roads.emplace_back(SerializeRoad(road));
    }

    json::array buildings;
    buildings.reserve(map.GetBuildings().size());

    for (const model::Building& building : map.GetBuildings()) {
        buildings.emplace_back(SerializeBuilding(building));
    }

    json::array offices;
    offices.reserve(map.GetOffices().size());

    for (const model::Office& office : map.GetOffices()) {
        offices.emplace_back(SerializeOffice(office));
    }

    return {
        {"id", *map.GetId()},
        {"name", map.GetName()},
        {"roads", std::move(roads)},
        {"buildings", std::move(buildings)},
        {"offices", std::move(offices)},
    };
}

StringResponse MakeMapsResponse(
    unsigned version,
    bool keep_alive) const {

    json::array maps;
    maps.reserve(game_.GetMaps().size());

```

```

        for (const model::Map& map : game_.GetMaps()) {
            maps.emplace_back(json::object{
                {"id", *map.GetId()},
                {"name", map.GetName()},
            });
        }

        return MakeJsonResponse(
            http::status::ok,
            version,
            keep_alive,
            std::move(maps));
    }

    static StringResponse MakeMapResponse(
        const model::Map& map,
        unsigned version,
        bool keep_alive) {

        return MakeJsonResponse(
            http::status::ok,
            version,
            keep_alive,
            SerializeMap(map));
    }
};

} // namespace http_handler

```

request_handler.cpp

```

#include "request_handler.h"

#include <cctype>
#include <string>
#include <string_view>

namespace http_handler {

RequestHandler::FileRequestResult RequestHandler::HandleFileRequest(
    const StringRequest& req) const {

    if (req.method() != http::verb::get &&
        req.method() != http::verb::head) {

        auto response = MakeTextResponse(
            http::status::method_not_allowed,
            req.version(),
            req.keep_alive(),
            "Only GET and HEAD methods are supported\n");

        response.set(http::field::allow, "GET, HEAD");
        return response;
    }
}

```

```

const beast::string_view target = req.target();

if (target.empty() || target.front() != '/') {
    return MakeTextResponse(
        http::status::bad_request,
        req.version(),
        req.keep_alive(),
        "Invalid request target\n");
}

const size_t query_position = target.find('?');
const beast::string_view encoded_path =
    target.substr(0, query_position);

const std::optional<std::string> decoded_path =
    UrlDecode(encoded_path);

if (!decoded_path) {
    return MakeTextResponse(
        http::status::bad_request,
        req.version(),
        req.keep_alive(),
        "Invalid URL encoding\n");
}

std::string relative_path = *decoded_path;

if (relative_path == "/") {
    relative_path = "/index.html";
}

relative_path.erase(0, 1);

fs::path requested_path =
    fs::weakly_canonical(static_root_ / relative_path);

if (!IsSubPath(requested_path, static_root_)) {
    return MakeTextResponse(
        http::status::bad_request,
        req.version(),
        req.keep_alive(),
        "Requested path is outside static directory\n");
}

boost::system::error_code ec;

if (fs::is_directory(requested_path, ec)) {
    requested_path /= "index.html";
    requested_path = fs::weakly_canonical(requested_path);
}

if (ec) {
    return MakeTextResponse(
        http::status::not_found,
        req.version(),
        req.keep_alive(),

```

```

        "File not found\n");
    }

    if (!IsSubPath(requested_path, static_root_)) {
        return MakeTextResponse(
            http::status::bad_request,
            req.version(),
            req.keep_alive(),
            "Requested path is outside static directory\n");
    }

    http::file_body::value_type file;

    file.open(
        requested_path.string().c_str(),
        beast::file_mode::read,
        ec);

    if (ec == beast::errc::no_such_file_or_directory) {
        return MakeTextResponse(
            http::status::not_found,
            req.version(),
            req.keep_alive(),
            "File not found\n");
    }

    if (ec) {
        return MakeTextResponse(
            http::status::internal_server_error,
            req.version(),
            req.keep_alive(),
            "Failed to open file\n");
    }

    const auto file_size = file.size();

    if (req.method() == http::verb::head) {
        EmptyResponse response{
            http::status::ok,
            req.version()
        };

        response.set(
            http::field::content_type,
            GetMimeType(requested_path));

        response.content_length(file_size);
        response.keep_alive(req.keep_alive());

        return response;
    }

    FileResponse response{
        std::piecewise_construct,
        std::make_tuple(std::move(file)),
        std::make_tuple(

```

```

        http::status::ok,
        req.version())
};

response.set(
    http::field::content_type,
    GetMimeType(requested_path));

response.content_length(file_size);
response.keep_alive(req.keep_alive());

return response;
}

RequestHandler::StringResponse RequestHandler::ReportServerError(
    unsigned version,
    bool keep_alive) const {

    return MakeErrorResponse(
        http::status::internal_server_error,
        version,
        keep_alive,
        "internalError",
        "Internal server error");
}

RequestHandler::StringResponse RequestHandler::HandleApiRequest(
    const StringRequest& request) {

    const std::string_view target{
        request.target().data(),
        request.target().size()
    };

    if (target == "/api/v1/game/join") {
        return HandleJoinGameRequest(request);
    }

    if (target == "/api/v1/game/players") {
        return HandlePlayersRequest(request);
    }

    if (target == "/api/v1/maps") {
        if (request.method() != http::verb::get) {
            return MakeErrorResponse(
                http::status::bad_request,
                request.version(),
                request.keep_alive(),
                "badRequest",
                "Bad request");
        }

        return MakeMapsResponse(
            request.version(),
            request.keep_alive());
    }
}

```

```

constexpr std::string_view kMapsPrefix = "/api/v1/maps/";

if (target.starts_with(kMapsPrefix)) {
    if (request.method() != http::verb::get) {
        return MakeErrorResponse(
            http::status::bad_request,
            request.version(),
            request.keep_alive(),
            "badRequest",
            "Bad request");
    }

    const std::string_view map_id =
        target.substr(kMapsPrefix.size());

    if (map_id.empty() ||
        map_id.find('/') != std::string_view::npos) {

        return MakeErrorResponse(
            http::status::bad_request,
            request.version(),
            request.keep_alive(),
            "badRequest",
            "Bad request");
    }

    const model::Map* map = game_.FindMap(
        model::Map::Id{std::string(map_id)});

    if (map == nullptr) {
        return MakeErrorResponse(
            http::status::not_found,
            request.version(),
            request.keep_alive(),
            "mapNotFound",
            "Map not found");
    }

    return MakeMapResponse(
        *map,
        request.version(),
        request.keep_alive());
}

return MakeErrorResponse(
    http::status::bad_request,
    request.version(),
    request.keep_alive(),
    "badRequest",
    "Bad request");
}

RequestHandler::StringResponse RequestHandler::HandleJoinGameRequest(
    const StringRequest& request) {

```

```

if (request.method() != http::verb::post) {
    auto response = MakeErrorResponse(
        http::status::method_not_allowed,
        request.version(),
        request.keep_alive(),
        "invalidMethod",
        "Only POST method is expected");

    response.set(http::field::allow, "POST");
    return response;
}

if (!IsJsonContentType(request)) {
    return MakeErrorResponse(
        http::status::bad_request,
        request.version(),
        request.keep_alive(),
        "invalidArgument",
        "Invalid content type");
}

try {
    const json::value request_json = json::parse(request.body());
    const json::object& object = request_json.as_object();

    const std::string user_name =
        json::value_to<std::string>(
            object.at("userName"));

    const std::string map_id =
        json::value_to<std::string>(
            object.at("mapId"));

    if (user_name.empty()) {
        return MakeErrorResponse(
            http::status::bad_request,
            request.version(),
            request.keep_alive(),
            "invalidArgument",
            "Invalid name");
    }

    try {
        const app::Application::JoinResult result =
            app_.JoinGame(
                model::Map::Id{map_id},
                user_name);

        return MakeJsonResponse(
            http::status::ok,
            request.version(),
            request.keep_alive(),
            json::object{
                {"authToken", *result.token},
                {"playerId", *result.player_id},
            });
    }
}

```

```

        } catch (const std::out_of_range&) {
            return MakeErrorResponse(
                http::status::not_found,
                request.version(),
                request.keep_alive(),
                "mapNotFound",
                "Map not found");
        }
    } catch (...) {
        return MakeErrorResponse(
            http::status::bad_request,
            request.version(),
            request.keep_alive(),
            "invalidArgument",
            "Join game request parse error");
    }
}

std::optional<model::Token> RequestHandler::ExtractBearerToken(
    const StringRequest& request) {

    constexpr std::string_view kBearerPrefix = "Bearer ";

    const auto authorization = request[http::field::authorization];

    const std::string_view value{
        authorization.data(),
        authorization.size()
    };

    if (!value.starts_with(kBearerPrefix)) {
        return std::nullopt;
    }

    const std::string_view token =
        value.substr(kBearerPrefix.size());

    if (token.size() != 32) {
        return std::nullopt;
    }

    for (const unsigned char ch : token) {
        if (!std::isxdigit(ch)) {
            return std::nullopt;
        }
    }

    return model::Token{std::string(token)};
}

RequestHandler::StringResponse RequestHandler::HandlePlayersRequest(
    const StringRequest& request) {

    if (request.method() != http::verb::get &&
        request.method() != http::verb::head) {

```

```

        auto response = MakeErrorResponse(
            http::status::method_not_allowed,
            request.version(),
            request.keep_alive(),
            "invalidMethod",
            "Invalid method");

        response.set(http::field::allow, "GET, HEAD");
        return response;
    }

    const std::optional<model::Token> token =
        ExtractBearerToken(request);

    if (!token) {
        return MakeErrorResponse(
            http::status::unauthorized,
            request.version(),
            request.keep_alive(),
            "invalidToken",
            "Authorization header is missing");
    }

    const model::Player* current_player =
        app_.FindPlayerByToken(*token);

    if (current_player == nullptr) {
        return MakeErrorResponse(
            http::status::unauthorized,
            request.version(),
            request.keep_alive(),
            "unknownToken",
            "Player token has not been found");
    }

    const model::GameSession& current_session =
        current_player->GetSession();

    json::object players_json;

    for (const auto& [player_id, player] : app_.GetPlayers()) {
        if (&player.GetSession() != &current_session) {
            continue;
        }

        players_json.emplace(
            std::to_string(*player_id),
            json::object{
                {"name", player.GetDog().GetName()}
            });
    }

    StringResponse response = MakeJsonResponse(
        http::status::ok,
        request.version(),
        request.keep_alive(),

```

```

        std::move(players_json));

    if (request.method() == http::verb::head) {
        response.body().clear();
        response.content_length(0);
    }

    return response;
}

} // namespace http_handler

```

main.cpp

```

#include <algorithm>
#include <csignal>
#include <cstdlib>
#include <filesystem>
#include <iostream>
#include <thread>
#include <vector>

#include <boost/asio.hpp>
#include <boost/json.hpp>
#include <boost/log/trivial.hpp>

#include "http_server.h"
#include "json_loader.h"
#include "logger.h"
#include "request_handler.h"

namespace fs = std::filesystem;
namespace net = boost::asio;
namespace json = boost::json;

using namespace std::literals;
using tcp = net::ip::tcp;

namespace {

net::io_context* g_ioc = nullptr;

void HandleSignal(int) {
    if (g_ioc != nullptr) {
        g_ioc->stop();
    }
}

template <typename Fn>
void RunWorkers(unsigned n, const Fn& fn) {
    n = std::max(1u, n);

    std::vector<std::thread> workers;
    workers.reserve(n - 1);
}

```

```

    while (--n) {
        workers.emplace_back(fn);
    }

    fn();

    for (std::thread& worker : workers) {
        worker.join();
    }
}

} // namespace

int main(int argc, const char* argv[]) {
    if (argc != 2) {
        std::cerr << "Usage: "sv << argv[^3_0] << " <config-file>\n"sv;
        return EXIT_FAILURE;
    }

    logger::InitLogger();

    try {
        const fs::path config_path = argv[^3_1];

        const fs::path static_root =
            config_path.parent_path().parent_path() / "static";

        model::Game game =
            json_loader::LoadGame(config_path.string());

        const unsigned num_threads =
            std::max(1u, std::thread::hardware_concurrency());

        net::io_context ioc{
            static_cast<int>(num_threads)
        };

        g_ioc = &ioc;

        std::signal(SIGINT, HandleSignal);
        std::signal(SIGTERM, HandleSignal);

        auto api_strand = net::make_strand(ioc);

        http_handler::RequestHandler handler{
            game,
            static_root,
            std::move(api_strand)
        };

        const auto address =
            net::ip::make_address("0.0.0.0");

        constexpr unsigned short port = 8080;

        http_server::ServeHttp(

```

```

        ioc,
        tcp::endpoint{address, port},
        handler);

BOOST_LOG_TRIVIAL(info)
    << logging::add_value(
        additional_data,
        json::object{
            {"port", port},
            {"address", address.to_string()},
        })
    << "server started";

logging::core::get()->flush();
std::cout.flush();

RunWorkers(num_threads, [&ioc] {
    ioc.run();
});

g_ioc = nullptr;

BOOST_LOG_TRIVIAL(info)
    << logging::add_value(
        additional_data,
        json::object{
            {"code", EXIT_SUCCESS},
        })
    << "server exited";

    return EXIT_SUCCESS;
} catch (const std::exception& ex) {
    g_ioc = nullptr;

    BOOST_LOG_TRIVIAL(error)
        << logging::add_value(
            additional_data,
            json::object{
                {"code", EXIT_FAILURE},
                {"exception", ex.what()},
            })
        << "server exited";

    return EXIT_FAILURE;
}
}

```

Перед сборкой

В вашем CMakeLists.txt должны компилироваться оба новых/изменённых .cpp:

```

add_executable(game_server
    main.cpp
    model.cpp
    json_loader.cpp

```

```
    request_handler.cpp
    http_server.cpp
    boost_json.cpp
)
```

`application.h` и `player.h` — header-only, поэтому отдельно в `add_executable` их добавлять не требуется.

После замены файлов удалите ошибочно добавленные обработчики из старого `main.cpp`: `HandleJoinGameRequest`, `ExtractBearerToken`, `HandlePlayersRequest`. В финальном `main.cpp` выше их нет: они должны жить исключительно в `RequestHandler`.^{[32] [34] [35]}

✱

прошу проверить

Проверил обновлённые файлы: направление верное — обработчики перенесены в `RequestHandler`, API маршрутизируется через `strand`, а собаки хранятся в `std::deque`. Но проект ещё не соберётся: остались несколько ошибок в шаблонных типах, `std::optional` и сигнатуре `HTTP-handler`.^{[37] [38] [39] [40] [41]}

Обязательные исправления

1. `application.h`: два места с `unique_ptr` и `make_unique`

Сейчас:

```
auto session = std::make_unique(&map);
```

и:

```
std::unordered_map<
    std::string,
    std::unique_ptr> sessions_;
```

Такой код не компилируется. Шаблонные параметры нужно указать явно:

```
auto session = std::make_unique<model::GameSession>(&map);
```

```
std::unordered_map<
    std::string,
    std::unique_ptr<model::GameSession>> sessions_;
```

Исправленный метод:

```

model::GameSession& GetOrCreateSession(const model::Map& map) {
    const std::string map_id = *map.GetId();

    if (auto it = sessions_.find(map_id); it != sessions_.end()) {
        return *it->second;
    }

    auto session = std::make_unique<model::GameSession>(&map);
    model::GameSession* result = session.get();

    sessions_.emplace(map_id, std::move(session));
    return *result;
}

```

Это критическая ошибка компиляции. ^[38]

2. request_handler.h: std::optional **обязан иметь параметр типа**

Сейчас объявлено:

```

static std::optional ExtractBearerToken(
    const StringRequest& request);

```

Нужно:

```

static std::optional<model::Token> ExtractBearerToken(
    const StringRequest& request);

```

std::optional — шаблон; написать его без <T> нельзя. ^[39]

3. request_handler.cpp: **исправьте определение** ExtractBearerToken

Сейчас:

```

std::optional RequestHandler::ExtractBearerToken(
    const StringRequest& request) {

```

Нужно:

```

std::optional<model::Token> RequestHandler::ExtractBearerToken(
    const StringRequest& request) {

```

Также ниже у вас STAD-форма:

```

const std::optional token =

```

```
ExtractBearerToken(request);
```

Она может поддерживаться в C++20, но здесь лучше не полагаться на вывод типа и написать явно:

```
const std::optional<model::Token> token =  
    ExtractBearerToken(request);
```

Это делает код переносимее и понятнее. ^[41]

4. request_handler.h: тип Strand нужно привести к фактическому executor

Сейчас:

```
using Strand = net::strand;
```

Если в исходном проекте определён точный alias в sdk.h, это может работать. Но для net::make_strand(ioc) корректным и стабильным типом является:

```
using Strand = net::strand<net::io_context::executor_type>;
```

Именно такой тип соответствует значению:

```
auto api_strand = net::make_strand(ioc);
```

в main.cpp. ^[37] ^[39]

5. request_handler.h: преобразование входящего request сделано рискованно

У вас handler принимает:

```
template <typename Body, typename Allocator, typename Send>  
void operator()(  
    tcp::endpoint,  
    http::request<Body, http::basic_fields<Allocator>>&& req,  
    Send&& send)
```

Но http_server.h вызывает обработчик в другой форме:

```
request_handler_(std::move(request_), SendLambda{*this});
```

То есть передаёт два аргумента, а ваш оператор требует три: tcp::endpoint + request + send.

В исходном `http_server.h` действительно вызов такой:

```
request_handler_(std::move(request_), SendLambda{*this});
```

Поэтому текущая сигнатура `RequestHandler::operator()` не будет найдена компилятором.
[\[42\]](#) [\[39\]](#)

Исправьте сигнатуру на:

```
template <typename Body, typename Allocator, typename Send>
void operator()(
    http::request<Body, http::basic_fields<Allocator>>&& req,
    Send&& send) {
```

То есть просто удалите параметр:

```
tcp::endpoint,
```

Иначе будет ошибка вида:

```
no match for call to '(http_handler::RequestHandler) (...)'
```

6. `request_handler.h`: `std::forward(req)` ИСПОЛЬЗОВАТЬ НЕЛЬЗЯ

Сейчас:

```
req = std::forward(req),
```

Это некорректно. `std::forward` всегда требует шаблонного параметра:

```
std::forward<T>(value)
```

В данном месте `request` уже является rvalue-reference, так что просто перемещайте его:

```
req = std::move(req),
```

Аналогично:

```
send(std::forward(result));
```

не скомпилируется. Должно быть:

```
send(std::forward<decltype(result)>(result));
```

Исправленный фрагмент:

```
auto handle = [  
    self = shared_from_this(),  
    req = std::move(req),  
    send,  
    version,  
    keep_alive  
]() mutable {  
    try {  
        assert(self->api_strand_.running_in_this_thread());  
  
        http::request<http::string_body> string_request{  
            std::move(req)  
        };  
  
        send(self->HandleApiRequest(string_request));  
    } catch (...) {  
        send(self->ReportServerError(version, keep_alive));  
    }  
};
```

И статические файлы:

```
std::visit(  
    [&send](auto&& result) {  
        send(std::forward<decltype(result)>(result));  
    },  
    HandleFileRequest(string_request));
```

В нынешней форме `std::forward(req)` и `std::forward(result)` — ошибки компиляции. ^[39]

7. request_handler.h: исправьте типы request и response

В выгруженном коде формат отображения скрывает шаблонные параметры, однако в реальном файле обязательно должны быть именно эти aliases:

```
using StringRequest = http::request<http::string_body>;  
using StringResponse = http::response<http::string_body>;  
using EmptyResponse = http::response<http::empty_body>;  
using FileResponse = http::response<http::file_body>;
```

Не оставляйте формы наподобие:

```
using StringRequest = http::request;  
using StringResponse = http::response;
```

`boost::beast::http::request` и `response` требуют `body type`.^[39]

8. `player.h`: используйте явный тип массива

Сейчас:

```
const std::array values{
    generator_(),
    generator_()
};
```

В C++20 `deduction guide` допустим, но в учебной сборке разумнее исключить неоднозначности:

```
const std::array<std::uint64_t, 2> values{
    generator_(),
    generator_()
};
```

Это не блокирующая ошибка при C++20, но практическая правка.^[40]

Исправленный `operator()`

Замените целиком публичный `operator()` в `RequestHandler` на ЭТОТ:

```
template <typename Body, typename Allocator, typename Send>
void operator()(
    http::request<Body, http::basic_fields<Allocator>>&& req,
    Send&& send) {

    const unsigned version = req.version();
    const bool keep_alive = req.keep_alive();

    try {
        if (IsApiRequest(req)) {
            auto handle = [
                self = shared_from_this(),
                req = std::move(req),
                send,
                version,
                keep_alive
           ]() mutable {
                try {
                    assert(self->api_strand_.running_in_this_thread());

                    http::request<http::string_body> string_request{
                        std::move(req)
                    };

                    send(self->HandleApiRequest(string_request));
                } catch (...) {

```

```

        send(self->ReportServerError(version, keep_alive));
    }
};

net::dispatch(api_strand_, std::move(handle));
return;
}

http::request<http::string_body> string_request{
    std::move(req)
};

std::visit(
    [&send](auto&& result) {
        send(std::forward<decltype(result)>(result));
    },
    HandleFileRequest(string_request));
} catch (...) {
    send(ReportServerError(version, keep_alive));
}
}

```

Это соответствует фактическому вызову из HttpSession:

```
request_handler_(std::move(request_), SendLambda{*this});
```

Важное исправление HEAD /players

Ваш текущий HEAD-код:

```
response.body().clear();
response.content_length(0);
```

возвращает пустое тело, что хорошо, но Content-Length: 0 технически не соответствует обычной семантике HEAD: заголовки должны соответствовать GET, а тело не отправляется.

Для учебного задания, вероятно, достаточно текущего поведения, поскольку сказано лишь, что GET и HEAD разрешены. Но лучше не очищать body вручную в StringResponse, потому что Beast всё равно формирует string-body response, а тест может сравнить длину.

Если требуется минимальная совместимость с тестами — оставьте текущий вариант. Если требуется HTTP-корректность — понадобится отдельный EmptyResponse для API HEAD, что потребует изменить возвращаемый тип API handler на variant. Это сейчас не обязательный рефакторинг. ^[41]

Логическая проблема ответа на неверный Content-Type

Задание явно требует Content-Type: application/json для join, но не задаёт точную структуру ошибки при другом media type. Ваш ответ:

```
{
  "code": "invalidArgument",
  "message": "Invalid content type"
}
```

логичен и использует 400. Однако многие автотесты проверяют только код 400, application/json, Cache-Control: no-cache и наличие согласованного error-объекта. Здесь решение выглядит разумно. ^[41] ^[39]

Что уже сделано правильно

- main.cpp стал чистым: HTTP-обработчики удалены, создан api_strand, а RequestHandler получает game, статическую директорию и strand. ^[37]
- MakeJsonResponse добавляет Content-Type: application/json, Cache-Control: no-cache и вызывает prepare_payload(), поэтому Content-Length рассчитывается автоматически. ^[39]
- /api/v1/game/join корректно проверяет POST, JSON Content-Type, JSON-body, userName, карту и создаёт ответ с authToken + playerId. ^[41]
- /api/v1/game/players допускает GET и HEAD, обрабатывает invalidToken и unknownToken, а список ограничивает игроками той же GameSession. ^[41]
- std::deque<Dog> устраняет инвалидирование Dog*, на который ссылается Player. ^[40]
- std::unordered_map<PlayerId, Player> позволяет хранить Player* в таблице токенов без инвалидирования при rehash. ^[40]

Итог

После исправления этих семи обязательных пунктов код будет значительно ближе к собираемому состоянию:

1. make_unique<model::GameSession>.
2. unique_ptr<model::GameSession>.
3. optional<model::Token> в .h.
4. optional<model::Token> в .cpp.
5. Удалить лишний tcp::endpoint из RequestHandler::operator().
6. Заменить неверные std::forward(req) / std::forward(result).
7. Убедиться в явных типах http::request<...> и http::response<...>.

После этого первым делом запустите сборку. Если она пройдёт, проверяйте по порядку: GET /api/v1/maps, затем успешный POST /join, затем GET /players с выданным токеном, затем негативные сценарии.

смысл операции

```
req = std::move(req)
```

`req = std::move(req)` в `capture`-списке лямбды означает: **переместить объект HTTP-запроса `req` внутрь создаваемой лямбды**, чтобы она владела им после выхода из `RequestHandler::operator()`.

В вашем случае это нужно потому, что API-запрос обрабатывается асинхронно через `net::dispatch(api_strand_, ...)`. Лямбда может выполняться позже, когда текущий вызов `operator()` уже завершился. ^[43]

Где это находится

Фрагмент выглядит так:

```
auto handle = [
    self = shared_from_this(),
    req = std::move(req),
    send,
    version,
    keep_alive
]() mutable {
    // обработка API
};
```

Левая часть:

```
req =
```

— имя нового поля внутри лямбды.

Правая часть:

```
std::move(req)
```

— исходный параметр функции `RequestHandler::operator()`.

То есть это не самоприсваивание. По смыслу это эквивалентно следующему классу:

```
class LambdaObject {
public:
    decltype(req) req; // собственный request внутри объекта лямбды
};
```

и его инициализации перемещением:

```
LambdaObject lambda_object{
    .req = std::move(req)
};
```

Почему не просто req

Если написать захват так:

```
[req]
```

то запрос будет скопирован внутрь лямбды.

HTTP-request содержит заголовки, строку target и, главное, body. Для POST /api/v1/game/join body содержит JSON. Копирование может быть заметно дороже, особенно для большого тела.

Вариант:

```
[req = std::move(req)]
```

переносит владение его внутренними ресурсами — строками и буферами — без лишнего копирования, когда типы это поддерживают.

Почему нельзя захватить по ссылке

Такой код был бы ошибочным:

```
auto handle = [&req] {
    Process(req);
};
```

или:

```
auto handle = [&] {
    Process(req);
};
```

Потому что req — параметр operator(). После возврата из функции он уничтожается.

Но задача отправляется в api_strand_:

```
net::dispatch(api_strand_, std::move(handle));
```

Когда strand выполнит лямбду позже, ссылка &req будет указывать на уже уничтоженный объект: это **висячая ссылка** и неопределённое поведение.

Перемещённый захват решает проблему времени жизни:

```
operator() (req)
|
├─ создаёт lambda, владеющую request
|   └─ req = std::move(req)
|
├─ передаёт lambda в api_strand
└─ завершается
```

Позже:

```
api_strand вызывает lambda
└─ lambda всё ещё владеет request
```

Зачем затем ещё один `std::move`

Внутри лямбды у вас находится:

```
http::request<http::string_body> string_request{
    std::move(req)
};
```

Здесь `req` — уже поле лямбды, а не исходный параметр функции.

Первый `move`:

```
req = std::move(req)
```

переносит `request` из `operator()` в объект лямбды.

Второй `move`:

```
std::move(req)
```

переносит `request` из объекта лямбды в локальную переменную `string_request`, которую принимает `HandleApiRequest`.

Полная цепочка владения:

```
HttpSession::request_
| std::move
▼
RequestHandler::operator() parameter req
| std::move
▼
поле req объекта lambda
| std::move
▼
string_request
```

```
| const StringRequest&  
▼  
HandleApiRequest(...)
```

Эта схема обеспечивает два свойства:

- request не копируется без необходимости;
- request гарантированно живёт до завершения API-обработчика. ^[44] ^[43]

Почему у лямбды есть mutable

По умолчанию оператор вызова лямбды является const. То есть захваченные по значению объекты внутри неё считаются константными.

Без mutable:

```
auto handle = [req = std::move(req)]() {  
    auto other = std::move(req); // нельзя или перемещение будет из const  
};
```

С mutable:

```
auto handle = [req = std::move(req)]() mutable {  
    auto other = std::move(req); // корректно  
};
```

Это необходимо, потому что внутри lambda нужно переместить её собственное поле req в string_request.

std::move **не перемещает сам по себе**

Важно: std::move не копирует и не «перекладывает» объект сам по себе. Это лишь приведение выражения к rvalue-ссылке, то есть сигнал компилятору:

«Этот объект больше не нужен в прежнем состоянии; можно выбрать move-конструктор или move-присваивание».

Фактическое перемещение происходит в месте инициализации:

```
req = std::move(req)
```

где лямбда инициализирует своё внутреннее поле из исходного request.

После операции исходный параметр req остаётся валидным, но его содержимое находится в неопределённом, но допустимом состоянии. В вашем коде это безопасно, потому что исходный req после постановки задачи больше не используется.

Короткий пример

```
void ProcessAsync(std::string&& text) {
    auto task = [text = std::move(text)]() mutable {
        std::cout << text << '\n';
    };

    PostToQueue(std::move(task));
}
```

Здесь:

1. text передаётся в ProcessAsync.
2. text = std::move(text) создаёт копию поля лямбды и перемещает в неё строковый буфер.
3. ProcessAsync завершается.
4. Позже очередь вызывает task.
5. Строка всё ещё существует, потому что принадлежит объекту лямбды.

Для request обработчика смысл абсолютно такой же: лямбда должна владеть запросом, пока его ожидает и выполняет api_strand.

✱✱

ТО ЕСТЬ

```
auto handle = self = shared_from_this(), req = std::move(req), send, version, keep_alive mutable { //
обработка API };
```

эквивалентно

```
auto handle = self = shared_from_this(), request = std::move(req), send, version, keep_alive mutable { //
обработка API };
```

Да, эквивалентно, если внутри тела лямбды вы используете соответствующее имя.

```
auto handle = [
    self = shared_from_this(),
    req = std::move(req),
    send,
    version,
    keep_alive
]() mutable {
    // Здесь req – поле (захваченная переменная) лямбды.
};
```

и:

```
auto handle = [
    self = shared_from_this(),
```

```
    request = std::move(req),
    send,
    version,
    keep_alive
]() mutable {
    // Здесь request — поле лямбды.
};
```

В обоих случаях справа берётся исходный параметр функции:

```
std::move(req)
```

а слева задаётся имя переменной, принадлежащей объекту лямбды:

```
req = ...
```

либо:

```
request = ...
```

Разница только в имени внутри лямбды

Первый вариант:

```
auto handle = [req = std::move(req)]() mutable {
    http::request<http::string_body> string_request{
        std::move(req)
    };
};
```

Второй вариант:

```
auto handle = [request = std::move(req)]() mutable {
    http::request<http::string_body> string_request{
        std::move(request)
    };
};
```

Второй вариант обычно читается проще, потому что он устраняет совпадение имён:

```
request = std::move(req)
^ поле lambda    ^ параметр operator()
```

Я бы рекомендовал именно его:

```

auto handle = [
    self = shared_from_this(),
    request = std::move(req),
    send,
    version,
    keep_alive
]() mutable {
    try {
        assert(self->api_strand_.running_in_this_thread());

        http::request<http::string_body> string_request{
            std::move(request)
        };

        send(self->HandleApiRequest(string_request));
    } catch (...) {
        send(self->ReportServerError(version, keep_alive));
    }
};

```

mutable всё ещё нужен: без него захваченное по значению поле `request` внутри лямбды считается `const`, и его нельзя корректно передать через `std::move` в новый `string_request`.

git.actions

Run `conan install . --output-folder=build --build=missing -s build_type=Release`

===== Input profiles =====

Profile host:

[settings]

arch=x86_64

build_type=Release

compiler=gcc

compiler.cppstd=gnu17

compiler.libcxx=libstdc++11

compiler.version=13

os=Linux

Profile build:

[settings]

arch=x86_64

```
build_type=Release
```

```
compiler=gcc
```

```
compiler.cppstd=gnu17
```

```
compiler.libcxx=libstdc++11
```

```
compiler.version=13
```

```
os=Linux
```

ERROR: Error while parsing [options] in conanfile.txt

Options should be specified as 'pkg/*:option=value'

Error: Process completed with exit code 1.

Ошибка означает, что в вашем `conanfile.txt` в секции `[options]` есть опция без области действия пакета — вероятно что-то вроде:

```
[options]
shared=False
```

В Conan 2 каждая option в потребительском `conanfile.txt` должна содержать имя пакета или паттерн пакета:

```
[options]
<package-pattern>:<option>=<value>
```

Например:

```
[options]
boost/*:shared=False
```

или для всех пакетов, у которых существует опция `shared`:

```
[options]
*:shared=False
```

Conan 2 документирует формат `[options]` как `package_name*:option=Value`; в качестве примеров используются `boost/*:shared=True` и `*:shared=True`.^[45]

Что проверить в conanfile.txt

Откройте файл и найдите блок:

```
[options]
```

Исправьте каждую строку, которая выглядит так:

```
option=value
```

на формат:

```
пакет/*:option=value
```

Типовые исправления

Неверно	Верно
shared=False	boost/*:shared=False
header_only=True	boost/*:header_only=True
without_test=True	boost/*:without_test=True
fPIC=True	some-package/*:fPIC=True
with_zlib=True	cpprestsdk/*:with_zlib=True

Если вы хотите задать shared=False всем зависимостям, которые поддерживают эту опцию:

```
[options]  
*:shared=False
```

Но для учебного сервера с Boost лучше задавать опции адресно, чтобы не сломать чужие зависимости:

```
[options]  
boost/*:shared=False
```

Пример корректного conanfile.txt

Для проекта на Boost/Beast/JSON минимальный файл может выглядеть так:

```
[requires]  
boost/1.86.0  
  
[generators]
```

```
CMakeDeps
CMakeToolchain

[options]
boost/*:shared=False
```

Если вы используете текущую версию Boost из другого задания, оставьте вашу версию в `[requires]`; менять её только из-за этой ошибки не требуется.

Почему GitHub Actions упал

Команда workflow:

```
conan install . \
  --output-folder=build \
  --build=missing \
  -s build_type=Release
```

корректна. Сбой произошёл раньше конфигурации CMake: Conan прочитал `conanfile.txt`, дошёл до `[options]` и отверг старый формат записи. В Conan 1 раньше встречалась запись вида `boost:shared=False`; для Conan 2 предпочтителен паттерн с версией/маской, например `boost/*:shared=False`.^[46]

Что сделать

1. Исправьте все строки секции `[options]` в `conanfile.txt`.
2. Закоммитьте изменение.
3. Повторно запустите GitHub Actions.

Локальная проверка:

```
rm -rf build
conan install . \
  --output-folder=build \
  --build=missing \
  -s build_type=Release
```

Если покажете содержимое вашего `conanfile.txt`, я укажу точную строку, которую нужно заменить.

✱

//conanfile.txt

```
[requires]
boost/1.79.0
miniaudio/0.11.9

[generators]
CMakeDeps
CMakeToolchain

[options]
boost:without_log=False
boost:without_thread=False
boost:without_date_time=False
boost:without_filesystem=False
boost:without_regex=False
boost:without_chrono=False
boost:without_atomic=False
```

В Conan 2 замените `boost:` на `boost/*:` во всех строках `[options]`. Сейчас у вас синтаксис Conan 1, а GitHub Actions использует Conan 2 — именно поэтому парсер сообщает, что ожидает `pkg/*:option=value`.^[55] ^[56]

Исправленный conanfile.txt

```
[requires]
boost/1.79.0
miniaudio/0.11.9

[generators]
CMakeDeps
CMakeToolchain

[options]
boost/*:without_log=False
boost/*:without_thread=False
boost/*:without_date_time=False
boost/*:without_filesystem=False
boost/*:without_regex=False
boost/*:without_chrono=False
boost/*:without_atomic=False
```

После этого команда должна пройти этап разбора recipe:

```
conan install . \
  --output-folder=build \
  --build=missing \
  -s build_type=Release
```

Можно упростить

Для вашего HTTP-сервера с Boost.Asio/Beast/JSON, судя по коду, вам точно нужны:

- `thread` — из-за `std::thread`, но это не обязательно `Boost.Thread`;
- `filesystem` — у вас применяется `std::filesystem`, не `Boost.Filesystem`;
- `log` — у вас подключается `boost/log/trivial.hpp`, поэтому `Boost.Log` действительно нужен;
- `chrono` и `atomic` обычно требуются транзитивно для `Boost.Log` и других компонентов.

Однако при установке Boost Conan обычно сам выбирает разумные значения, и `without_*` часто вовсе не обязательно прописывать. Минимальный вариант можно сделать таким:

```
[requires]
boost/1.79.0
miniaudio/0.11.9

[generators]
CMakeDeps
CMakeToolchain
```

Если ваш `CMakeLists.txt` явно ищет конкретные компоненты Boost, например:

```
find_package(Boost REQUIRED COMPONENTS log log_setup thread filesystem regex chrono)
```

то оставьте исправленный блок `[options]`, чтобы гарантированно собрать эти компоненты.

Важное замечание про Boost 1.79.0

В Conan 2 опция `boost/*:without_log=False` означает: **не отключать** компонент `Boost.Log`. Это именно то, что нужно, если у вас есть:

```
#include <boost/log/trivial.hpp>
```

Аналогично, `boost/*:without_thread=False` включает `Boost.Thread`, `boost/*:without_regex=False` — `Boost.Regex` и т. д. ^[55]

После исправления

Проверьте ещё раз локально:

```
rm -rf build
conan install . \
  --output-folder=build \
  --build=missing \
  -s build_type=Release
```

Если следующий сбой будет уже на `cmake --build`, пришлите полный лог ошибки вместе с `CMakeLists.txt`: тогда можно будет проверить, совпадают ли компоненты из `find_package(Boost COMPONENTS ...)` с настройками Conan.

✱✱

Annotations

1 warning and 1 notice

build

Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/cache@v4, actions/checkout@v4, actions/setup-python@v5. For more information see: <https://github.blog/changelog/2025-09-19-deprecation-of-node-20-on-github-actions-runners/>

build

"The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more information, see <https://github.com/actions/runner-images/issues/14748>"

Это не ошибка вашей сборки, а два информационных предупреждения GitHub Actions. Они не являются причиной падения `conan install`; причина падения была в синтаксисе `[options]` в `conanfile.txt`.

Предупреждение Node.js 20

Сообщение:

```
Node.js 20 is deprecated.
actions/cache@v4, actions/checkout@v4, actions/setup-python@v5
are being forced to run on Node.js 24
```

означает:

- GitHub Actions принудительно запускает JavaScript actions через Node.js 24;
- workflow продолжает работать;
- указанные action-версии пока содержат metadata с Node.js 20;
- GitHub постепенно мигрирует runtime actions на Node.js 24.

GitHub указывает, что Node.js 20 достиг EOL в апреле 2026 года; с 16 июня 2026 GitHub-hosted runners начали предпочитать Node.js 24, а удаление Node.js 20 было запланировано на 23 сентября 2026. Для пользователей workflow основная рекомендация — обновлять actions до их актуальных версий.^[57]

Что делать

Если workflow содержит что-то вроде:

```
- uses: actions/checkout@v4
- uses: actions/setup-python@v5
- uses: actions/cache@v4
```

то прямо сейчас это не нужно исправлять для прохождения сборки. Ваш workflow выполняется, потому что GitHub уже принудительно переключил runtime на Node.js 24.

Позже обновите версии actions, когда GitHub выпустит новые major/minor версии с Node 24 runtime. Не нужно добавлять:

```
ACTIONS_ALLOW_USE_UNSECURE_NODE_VERSION: true
```

Это было только временным способом удержаться на устаревшем Node 20 и вам не требуется.

Если хотите заранее явно включить новый runtime, можно добавить на уровне workflow:

```
env:
  FORCE_JAVASCRIPT_ACTIONS_TO_NODE24: true
```

Но предупреждение уже говорит, что GitHub и так запускает actions на Node.js 24, поэтому практически это ничего не изменит.

Предупреждение Ubuntu

Сообщение:

```
"The ubuntu-latest label will migrate to Ubuntu 26
beginning October 19, 2026."
```

означает, что тег:

```
runs-on: ubuntu-latest
```

сейчас указывает на текущий образ GitHub, но начиная с 19 октября 2026 начнёт мигрировать на Ubuntu 26.

Это не падение workflow, а уведомление о будущем изменении среды. После миграции потенциально могут измениться:

- версии gcc, cmake, Python и системных библиотек;
- набор предустановленных пакетов;

- поведение Conan при поиске или сборке отсутствующих бинарных пакетов;
- ABI и базовый образ, используемый cache-ключами.

Что выбрать

Цель	Рекомендация
Просто проходить CI сейчас	Оставить <code>runs-on: ubuntu-latest</code>
Воспроизводимая среда и защита от неожиданной миграции	Закрепить <code>runs-on: ubuntu-24.04</code>
Ранняя проверка совместимости с Ubuntu 26	Добавить отдельную CI job на <code>ubuntu-26.04</code> , когда образ станет доступен
Стабильная production-like CI-среда	Пиновать образ на конкретную версию Ubuntu

Для учебного проекта я бы использовал зафиксированный runner:

```
runs-on: ubuntu-24.04
```

Тогда переход `ubuntu-latest` на Ubuntu 26 не сможет неожиданно сломать ваш проект, например из-за нового GCC, CMake или зависимости Conan.

Рекомендуемый workflow-фрагмент

```
name: Build

on:
  push:
  pull_request:

jobs:
  build:
    runs-on: ubuntu-24.04

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install Conan
        run: pip install conan

      - name: Create Conan profile
        run: conan profile detect --force

      - name: Install dependencies
        run: >
```

```

        conan install .
        --output-folder=build
        --build=missing
        -s build_type=Release

- name: Configure CMake
  run: >
    cmake -S . -B build
    -DCMAKE_TOOLCHAIN_FILE=build/conan_toolchain.cmake
    -DCMAKE_BUILD_TYPE=Release

- name: Build
  run: cmake --build build --config Release --parallel

```

Главное сейчас

Ваш следующий commit должен исправить conanfile.txt:

```

[options]
boost/ *:without_log=False
boost/ *:without_thread=False
boost/ *:without_date_time=False
boost/ *:without_filesystem=False
boost/ *:without_regex=False
boost/ *:without_chrono=False
boost/ *:without_atomic=False

```

После этого Node.js и Ubuntu annotations можно считать второстепенными: они не должны препятствовать сборке.

✱

PS C:\Users\ISB\source\repos\game_server_trivial> git switch main

Switched to branch 'main'

Your branch is behind 'origin/main' by 4 commits, and can be fast-forwarded.

(use "git pull" to update your local branch)

PS C:\Users\ISB\source\repos\game_server_trivial> git merge log

Updating e7da323..5fe8e15

Fast-forward

.github/workflows/build.yml | 40 +++

README.md | 72 +++-

conanfile.txt | 14 +-

data/config.json | 31 +-

src/application.h | 77 ++++++

src/http_server.h | 2 +-

src/main.cpp | 189 ++++++-----

src/player.h | 191 ++++++++

```
src/request_handler.cpp | 424 ++++++-----
src/request_handler.h | 787 ++++++-----
static/abc..html | 11 +
static/file with spaces.html | 11 +
static/index.html | 3 +
13 files changed, 1342 insertions(+), 510 deletions(-)
create mode 100644 .github/workflows/build.yml
create mode 100644 src/application.h
create mode 100644 src/player.h
create mode 100644 static/abc..html
create mode 100644 static/file with spaces.html
PS C:\Users\ISB\source\repos\game_server_trivial> git commit -m "Pullrequest 2026.09.23_19:36"
On branch main
Your branch and 'origin/main' have diverged,
and have 5 and 4 different commits each, respectively.
```

nothing to commit, working tree clean

```
PS C:\Users\ISB\source\repos\game_server_trivial> git push
```

У вас сейчас две независимые линии истории main: локально есть 5 коммитов, которых нет на origin/main, а на сервере — 4 коммита, которых нет у вас. Поэтому git commit ничего не делает: рабочая директория чистая, но историю веток ещё нужно объединить.

Судя по вашему выводу, команда:

```
git merge log
```

не выполнила merge ветки log, а Git интерпретировал это как сокращённую форму команды с ревизией/путём или вы не показали саму цель merge. Результат Updating e7da323..5fe8e15 Fast-forward означает, что указатель локальной ветки был просто передвинут вперёд, но затем Git увидел расхождение с актуальным origin/main.

Что сделать безопасно

Сначала обновите информацию об удалённом репозитории и посмотрите, какие именно коммиты отличаются:

```
git fetch origin

git log --graph --oneline --decorate --all -20

git log --oneline origin/main..HEAD
git log --oneline HEAD..origin/main
```

Значение последних двух команд:

```
origin/main..HEAD
```

— локальные коммиты, которых нет на GitHub.

```
HEAD..origin/main
```

— коммиты на GitHub, которых нет локально.

Расхождение веток как раз означает, что обе стороны имеют разные коммиты; Git не может безопасно сделать обычный push без предварительной интеграции удалённой истории. ^[58] ^[59]

Рекомендуемый вариант: rebase

Если это ваш личный репозиторий и локальные 5 коммитов ещё не опубликованы, используйте:

```
git pull --rebase origin main
```

Эта команда:

1. Заберёт 4 коммита с origin/main.
2. Временно уберёт ваши 5 локальных коммитов.
3. Передвинет локальный main на актуальный origin/main.
4. Повторно применит ваши 5 коммитов поверх него.
5. Создаст линейную историю без merge-коммита.

После успешного rebase:

```
git push origin main
```

`git pull --rebase` — стандартный вариант сохранения обеих линий коммитов без лишнего merge-коммита. ^[59]

Если возникнут конфликты

Git остановится и укажет конфликтующие файлы.

Проверьте состояние:

```
git status
```

После ручного исправления каждого конфликта:

```
git add <исправленный-файл>
git rebase --continue
```

Например:

```
git add conanfile.txt src/request_handler.h src/request_handler.cpp
git rebase --continue
```

Если нужно полностью отменить rebase и вернуться к состоянию до команды:

```
git rebase --abort
```

После окончания:

```
git status
git log --graph --oneline --decorate -15
git push origin main
```

Альтернатива: merge

Если хотите сохранить явно видимый merge-коммит, выполните:

```
git pull --no-rebase origin main
git push origin main
```

При конфликте порядок такой же:

```
git status
# исправить конфликтующие файлы
git add <файлы>
git commit
git push origin main
```

GitHub рекомендует сначала забрать и объединить изменения с remote (`git fetch + git merge` или `git pull`), поскольку push, который не является fast-forward, может быть отклонён во избежание потери истории. ^[58]

Чего не делать

Не выполняйте:

```
git push --force
```

и тем более:

```
git push --force origin main
```

Пока вы не изучили обе группы коммитов. Такой push может переписать удалённую ветку и удалить 4 удалённых коммита из её обычной истории.

После `git pull --rebase origin main` обычного `git push` должно быть достаточно, потому что ваши локальные коммиты до этого не были опубликованы.

Самая короткая последовательность

Если вы хотите сохранить все изменения и у вас нет незакоммиченных файлов:

```
git fetch origin
git pull --rebase origin main
git push origin main
```

Если `git pull --rebase` завершится без конфликтов — на этом всё.

✱✱

```
PS C:\Users\ISB\source\repos\game_server_trivial> git
pull --rebase origin main
```

```
error: Pulling is not possible because you have unmerged files.
hint: Fix them up in the work tree, and then use 'git add/rm <file>'
hint: as appropriate to mark resolution and make a commit.
fatal: Exiting because of an unresolved conflict.
PS C:\Users\ISB\source\repos\game_server_trivial>
```

У вас уже начат merge с конфликтом — вероятнее всего, после предыдущей команды `git merge log`. Пока этот конфликт не будет завершён или отменён, Git не позволит запускать `pull --rebase`.

Сначала выясните, какие именно файлы конфликтуют:

```
git status
```

Он покажет блок наподобие:

```
Unmerged paths:
  both modified: src/request_handler.h
  both modified: conanfile.txt
```

Git ожидает один из двух осознанных вариантов: завершить текущий merge либо отменить его. Конфликтующие файлы нужно отредактировать, добавить через `git add`, а затем зафиксировать merge-коммит. ^[68] ^[69]

Рекомендуемый путь: завершить текущий merge

Поскольку вы уже сделали merge и в нём есть конфликт, не нужно начинать rebase поверх незавершённого merge.

1. Посмотрите конфликтующие файлы

```
git status
```

Также можно найти все конфликтные маркеры:

```
git diff --name-only --diff-filter=U
git grep -n -e '<<<<<<<' -e '=====' -e '>>>>>>>'
```

2. Разрешите конфликт в каждом файле

В конфликтном файле Git добавляет блок такого вида:

```
<<<<<<< HEAD
// Ваша локальная версия
=====
// Версия из ветки/remote, которую вы merge-ите
>>>>>>> origin/main
```

Нужно:

1. Оставить правильный итоговый код — вашу версию, удалённую версию или объединённый вариант.
2. Полностью удалить строки:
 - <<<<<<< HEAD
 - =====
 - >>>>>>> ...

3. Сохранить файл.

Для проекта я бы внимательно объединил изменения в следующих потенциально конфликтных файлах:

```
conanfile.txt
src/request_handler.h
src/request_handler.cpp
src/main.cpp
src/player.h
src/application.h
.github/workflows/build.yml
```

В частности, в conanfile.txt оставьте Conan 2-синтаксис:

```
[options]
boost/*:without_log=False
boost/*:without_thread=False
boost/*:without_date_time=False
boost/*:without_filesystem=False
boost/*:without_regex=False
boost/*:without_chrono=False
boost/*:without_atomic=False
```

3. Пометьте конфликты как разрешённые

После редактирования:

```
git add conanfile.txt
git add src/request_handler.h
git add src/request_handler.cpp
git add src/main.cpp
git add src/player.h
git add src/application.h
git add .github/workflows/build.yml
```

Добавляйте только файлы, которые реально отображает `git status` как конфликтующие. Можно добавить всё сразу, если вы проверили изменения:

```
git add .
```

Перед коммитом полезно проверить staged-результат:

```
git diff --cached
```

И убедиться, что `conflict markers` исчезли:

```
git grep -n -e '<<<<<<<' -e '=====' -e '>>>>>>>'
```

Если команда не выводит ничего — маркеров нет.

4. Завершите merge

```
git commit -m "Merge remote changes into main"
```

После этого:

```
git status
git fetch origin
git push origin main
```

Если на удалённом `main` снова появились новые коммиты, Git может снова отклонить `push`. Тогда выполните:

```
git pull --rebase origin main
git push origin main
```

Но только после завершения текущего `merge`.

Альтернатива: отменить текущий `merge`

Если вы не хотите разбирать незавершённый `merge` и хотите вернуться к состоянию до него:

```
git merge --abort
```

Затем:

```
git status
git fetch origin
git pull --rebase origin main
```

Если `rebase` остановится на конфликтах:

```
git status

# Исправить конфликты вручную.
git add <исправленные-файлы>

git rebase --continue
```

Если передумаете и захотите отменить `rebase`:

```
git rebase --abort
```

Когда выбрать отмену

Используйте `git merge --abort`, если:

- `merge` был случайным;
- вы не понимаете, какую ветку реально пытались влить;
- хотите аккуратную линейную историю через `rebase`;
- локальные 5 коммитов хотите применить поверх актуального `origin/main`.

Команда, которую выполните прямо сейчас

Начните только с неё:

```
git status
```

Затем покажите её вывод. По нему можно точно сказать:

- какой процесс активен: merge, rebase, cherry-pick или другой;
- какие файлы конфликтуют;
- какую сторону конфликта нужно сохранить;
- какие конкретные `git add` / `git rm` использовать.

✱

PS C:\Users\ISB\source\repos\game_server_trivial> git status

On branch main

Your branch and 'origin/main' have diverged,

and have 5 and 4 different commits each, respectively.

(use "git pull" if you want to integrate the remote branch with yours)

You have unmerged paths.

(fix conflicts and run "git commit")

(use "git merge --abort" to abort the merge)

Unmerged paths:

(use "git add <file>..." to mark resolution)

both added: .github/workflows/build.yml

no changes added to commit (use "git add" and/or "git commit -a")

PS C:\Users\ISB\source\repos\game_server_trivial>

Конфликт только один: .github/workflows/build.yml. Это конфликт типа **both added**: файл был независимо создан и в вашей локальной истории, и в удалённой истории. Остальные файлы Git объединил автоматически.

Сейчас нужно выбрать или объединить две версии workflow, добавить итоговый файл в index и завершить уже начатый merge. После этого можно будет сделать обычный `git push`. [\[78\]](#) [\[79\]](#)

1. Посмотрите обе версии

В PowerShell выполните:

```
git diff -- .github/workflows/build.yml
```

Вы увидите примерно такой блок:

```
<<<<<<< HEAD
# Ваша локальная версия workflow
=====
# Версия из origin/main
>>>>>>> ...
```

Либо можно вывести каждую версию отдельно без ручного поиска маркеров.

Ваша локальная версия (HEAD):

```
git show :2:.github/workflows/build.yml
```

Версия, которую Git пытается влить:

```
git show :3:.github/workflows/build.yml
```

Для конфликта типа `both added`:

- `:2:` — версия текущей ветки `main` до `merge`, то есть ваша локальная.
- `:3:` — версия вливаемой ветки, то есть `remote`-история, которую вы объединили.

2. Практичное решение

Скорее всего, вам нужен один `workflow`, а не два. Откройте файл:

```
notepad .github/workflows/build.yml
```

И оставьте одну корректную версию.

Для вашего проекта важно, чтобы она содержала `Conan 2 install`, `CMake Toolchain` и сборку. Базовый рабочий вариант:

```
name: Build

on:
  push:
  pull_request:

jobs:
```

```

build:
  runs-on: ubuntu-24.04

  steps:
    - name: Checkout
      uses: actions/checkout@v4

    - name: Set up Python
      uses: actions/setup-python@v5
      with:
        python-version: "3.12"

    - name: Install Conan
      run: |
        python -m pip install --upgrade pip
        pip install conan

    - name: Create Conan profile
      run: conan profile detect --force

    - name: Install dependencies
      run: |
        conan install . ^
        --output-folder=build ^
        --build=missing ^
        -s build_type=Release

    - name: Configure CMake
      run: |
        cmake -S . -B build ^
        -DCMAKE_TOOLCHAIN_FILE=build/conan_toolchain.cmake ^
        -DCMAKE_BUILD_TYPE=Release

    - name: Build
      run: cmake --build build --config Release --parallel

```

Но в YAML на Linux runner нельзя использовать PowerShell continuation ^. GitHub Actions run по умолчанию запускает Bash. Поэтому используйте именно этот вариант:

```

name: Build

on:
  push:
  pull_request:

jobs:
  build:
    runs-on: ubuntu-24.04

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5

```

```

with:
  python-version: "3.12"

- name: Install Conan
  run: |
    python -m pip install --upgrade pip
    pip install conan

- name: Create Conan profile
  run: conan profile detect --force

- name: Install dependencies
  run: |
    conan install . \
      --output-folder=build \
      --build=missing \
      -s build_type=Release

- name: Configure CMake
  run: |
    cmake -S . -B build \
      -DCMAKE_TOOLCHAIN_FILE=build/conan_toolchain.cmake \
      -DCMAKE_BUILD_TYPE=Release

- name: Build
  run: cmake --build build --config Release --parallel

```

Если в одной из ваших двух версий workflow уже есть дополнительные важные шаги — например, ctest, clang-format, upload artifacts или cache Conan — перенесите их в итоговый файл.

3. Убедитесь, что маркеров больше нет

После сохранения:

```
git grep -n -e '<<<<<<<' -e '=====' -e '>>>>>>>'
```

Нормальный результат — **нет вывода**.

Проверьте YAML как текст:

```
Get-Content .github/workflows/build.yml
```

4. Завершите merge

```
git add .github/workflows/build.yml
git status
```

После `git add` строка `both added` должна исчезнуть. Затем:

```
git commit -m "Merge remote changes into main"
```

Проверьте:

```
git status
```

Ожидаемо:

```
On branch main
Your branch is ahead of 'origin/main' by ... commits.
nothing to commit, working tree clean
```

Теперь отправьте результат:

```
git push origin main
```

Короткая последовательность

Если хотите оставить итоговый workflow после ручной проверки:

```
notepad .github/workflows/build.yml

git add .github/workflows/build.yml

git commit -m "Merge remote changes into main"

git push origin main
```

Не выполняйте сейчас `git pull --rebase` и не используйте `git push --force`: сначала завершите текущий merge, начатый ранее.



моргает двоеточие и diff ждёт моих действий

Вы сейчас, вероятно, находитесь внутри pager-программы `less`, которую Git открыл для просмотра `git diff`. Мигающее двоеточие — это режим ввода команды `less`, а не запрос Git на редактирование файла.

Чтобы просто выйти из просмотра `diff`, нажмите:

```
q
```

Вернётесь в PowerShell.

Полезные клавиши в `git diff`

- `q` — выйти.
- `Space` или `Page Down` — вниз на страницу.
- `b` или `Page Up` — вверх на страницу.
- `↑/↓` — прокрутить строку.
- `/текст` — найти текст.
- `n` — перейти к следующему совпадению.
- `g` — в начало.
- `G` — в конец.

Если вы уже нажали `:` и видите приглашение внизу, можно:

```
Esc
```

а затем:

```
q
```

или иногда сразу:

```
q
```

Что сделать дальше

После выхода откройте конфликтный YAML именно для редактирования:

```
notepad .github/workflows/build.yml
```

Если в файле есть блоки:

```
<<<<<<< HEAD
...
=====
...
>>>>>>> ...
```

удалите маркеры и оставьте один итоговый рабочий workflow.

После сохранения:

```
git add .github/workflows/build.yml
git commit -m "Merge remote changes into main"
```

```
git push origin main
```

Конфликт был только в `.github/workflows/build.yml`; пока вы не сделаете `git add` после ручного выбора итоговой версии, Git считает merge незавершённым.

thank you

Пожалуйста. После q выйдете из просмотра diff, разрешите конфликт в `.github/workflows/build.yml`, затем выполните:

```
git add .github/workflows/build.yml
git commit -m "Merge remote changes into main"
git push origin main
```

Если появится конфликт или ошибка сборки в GitHub Actions — пришлите вывод, разберём по шагам.

1. model.h
2. player.h
3. request_handler.h
4. main.cpp
5. logger.h
6. model.cpp
7. request_handler.cpp
8. tagged.h
9. sdk.h
10. boost_json.cpp
11. http_server.cpp
12. json_loader.cpp
13. json_loader.h
14. http_server.h
15. application.h
16. main.cpp
17. player.h
18. request_handler.h
19. request_handler.cpp
20. json_loader.cpp
21. json_loader.h
22. logger.h
23. model.cpp
24. model.h

25. sdk.h
26. tagged.h
27. boost_json.cpp
28. http_server.cpp
29. boost_json.cpp
30. http_server.h
31. application.h
32. main.cpp
33. player.h
34. request_handler.h
35. request_handler.cpp
36. boost_json.cpp
37. main.cpp
38. application.h
39. request_handler.h
40. player.h
41. request_handler.cpp
42. http_server.h
43. request_handler.h
44. http_server.h
45. https://docs.conan.io/2/reference/conanfile_txt.html
46. <https://github.com/conan-io/conan/issues/13417>
47. <https://docs.conan.io/en/1.5/reference/conanfile/attributes.html>
48. <https://docs.conan.io/en/1.8/reference/conanfile/attributes.html>
49. <https://docs.conan.io/en/1.16/reference/conanfile/attributes.html>
50. <https://docs.conan.io/2/reference/conanfile/attributes.html>
51. <https://docs.conan.io/1/reference/conanfile/attributes.html>
52. <https://docs.conan.io/en/1.4/reference/conanfile/attributes.html>
53. https://docs.conan.io/1/reference/conanfile_txt.html
54. <https://docs.conan.io/en/1.6/reference/conanfile/attributes.html>
55. https://docs.conan.io/2/reference/conanfile_txt.html
56. <https://github.com/conan-io/conan/issues/13417>
57. <https://github.blog/changelog/2025-09-19-deprecation-of-node-20-on-github-actions-runners/>
58. <https://docs.github.com/en/get-started/using-git/dealing-with-non-fast-forward-errors>
59. <https://www.git-tower.com/learn/git/faq/you-have-divergent-branches>
60. <https://www.geeksforgeeks.org/git/what-to-do-when-git-branch-has-diverged/>
61. <https://jvns.ca/blog/2024/02/01/dealing-with-diverged-git-branches/>
62. <https://www.git-tower.com/learn/git/faq/git-fast-forward>

63. <https://oneuptime.com/blog/post/2026-01-24-git-divergent-branches/view>
64. <https://jvns.ca/blog/2024/04/10/notes-on-git-error-messages/>
65. <https://stackoverflow.com/questions/62653114/how-can-i-deal-with-this-git-warning-pulling-without-specifying-how-to-reconcile>
66. <https://stackoverflow.com/questions/8975925/on-diverged-git-branches/8976113>
67. <https://stackoverflow.com/questions/71768999/how-to-merge-when-you-get-error-hint-you-have-divergent-branches-and-need-to-s>
68. <https://www.atlassian.com/git/tutorials/using-branches/merge-conflicts>
69. <https://git-scm.com/docs/git-commit/2.31.0>
70. <https://stackoverflow.com/questions/3207029/how-to-resolve-git-status-unmerged-paths>
71. <https://docs.codeberg.org/collaborating/resolve-conflicts/>
72. <https://www.cs.swarthmore.edu/git/git-merge.php>
73. <https://www.git-tower.com/learn/git/faq/solve-merge-conflicts>
74. <https://www.git-tower.com/learn/git/ebook/en/command-line/advanced-topics/merge-conflicts>
75. <https://opensource.com/article/20/4/git-merge-conflict>
76. https://githowto.com/resolving_conflicts
77. <https://apxml.com/courses/getting-started-with-git/chapter-4-branching-merging-basics/handling-merge-conflicts>
78. <https://docs.codeberg.org/collaborating/resolve-conflicts/>
79. <https://git-scm.com/docs/git-commit/2.31.0>